

PetClinic

Sistema de Gestión de Clínica Veterinaria

Documentación Técnica del Proyecto

Departamento de Desarrollo

25 de junio de 2026

Índice general

1. Introducción	8
1.1. Descripción del Proyecto	8
1.1.1. Objetivos del Sistema	8
1.1.2. Público Objetivo	9
1.2. Tecnologías Utilizadas	9
1.2.1. Lenguaje y Framework	9
1.2.2. Base de Datos	9
1.2.3. Seguridad	9
1.2.4. Frontend	10
1.2.5. Otras Tecnologías	10
1.3. Convenciones del Proyecto	10
1.4. Requisitos del Sistema	10
2. Arquitectura del Sistema	12
2.1. Visión General	12
2.1.1. Diagrama de Capas	13
2.2. Flujo de una Petición	13
2.3. Flujo de Notificaciones en Tiempo Real (WebSocket)	14
2.4. Patrones de Diseño	15
2.4.1. Patrones Implementados	15
2.5. Diagrama de Componentes	16
2.5.1. Capas del Sistema	16
2.6. Esquema de Navegación	16
3. Modelo de Datos	18
3.1. Diagrama Entidad-Relación	18
3.1.1. Entidades del Sistema	18
3.1.2. Enumeraciones	20
3.1.3. Relaciones Principales	21
3.1.4. Listado Completo de Repositorios	21

4. Backend	23
4.1. Estructura del Proyecto	23
4.2. Controladores REST	24
4.2.1. Autenticación	24
4.2.2. Gestión Clínica	24
4.2.3. Portal Cliente	25
4.2.4. Público	25
4.2.5. Pagos Online	25
4.2.6. Auditoría	25
4.3. Capa de Servicios	26
4.3.1. Ejemplo: Validación de Solapamiento de Citas	26
4.3.2. Ejemplo: Generación de PDF con iText 7	27
4.3.3. Ejemplo: Integración con Stripe	27
4.4. Configuración	28
4.4.1. Application Properties	28
4.4.2. DataInitializer	29
4.5. Manejo de Excepciones	29
5. Frontend	31
5.1. Arquitectura del Frontend	31
5.1.1. Estructura de Archivos	31
5.2. Pantalla de Login	31
5.3. Dashboard	32
5.4. Agenda de Citas	33
5.5. Gestión de Clientes y Mascotas	35
5.6. Facturación	35
5.7. Reportes	37
5.8. Odontograma Canino	38
5.9. Portal del Cliente	39
5.10. Sistema de Temas	40
5.11. API Cliente (api.js)	41
6. Seguridad	43
6.1. Autenticación JWT	43
6.1.1. JwtTokenProvider	43
6.1.2. JwtAuthFilter	44
6.2. Control de Acceso por Roles	45
6.2.1. SecurityConfig	45
6.3. CustomUserDetailsService	46
6.4. Auditoría	47
6.5. Protección de Contraseñas	48
6.6. CORS	49
6.7. Protección CSRF	49

7. Módulos del Sistema	50
7.1. Visión General	50
7.2. Módulo de Veterinarios	50
7.3. Módulo de Clientes	51
7.4. Módulo de Mascotas	51
7.5. Módulo de Citas	52
7.6. Módulo de Facturación	52
7.7. Módulo de Servicios	52
7.8. Módulo de Medicamentos e Inventario	53
7.9. Módulo de Historial Médico	54
7.10. Módulo de Vacunas	55
7.11. Módulo de Hospitalización	56
7.12. Módulo de Planes de Tratamiento	57
7.13. Módulo de Consentimientos Informados	58
7.14. Módulo de Odontograma	59
7.15. Módulo de Pagos Online (Stripe)	59
7.16. Módulo de Reserva Pública	60
7.17. Módulo de Reportes	60
7.18. Módulo de Auditoría	60
7.19. Módulo de Configuración	61
7.20. Módulo de Notificaciones	62
7.21. Sistema de Temas	63
7.22. Credenciales de Prueba	63
8. Pruebas	64
8.1. Enfoque de Pruebas	64
8.2. Ejecución de Pruebas	64
8.3. Clases de Prueba	65
8.3.1. VeterinarySystemIntegrationTest	65
8.3.2. NewFeaturesIntegrationTest	66
8.4. Configuración de Pruebas	67
8.5. Ejemplo: Prueba de Creación de Factura con Descuento de Stock	67
8.6. Ejemplo: Prueba de Endpoints Públicos	68
8.7. Cobertura de Pruebas	68
9. Despliegue	70
9.1. Requisitos del Servidor	70
9.2. Configuración de Base de Datos	70
9.2.1. Crear Base de Datos PostgreSQL	70
9.2.2. Verificar Conexión	70
9.3. Ejecución en Desarrollo	71
9.4. Compilación para Producción	71
9.5. Despliegue con Docker	71

9.5.1. Dockerfile	71
9.5.2. Docker Compose	72
9.6. Despliegue con systemd (Producción Linux)	72
9.7. Proxy Inverso con Nginx	73
9.8. Variables de Entorno	74
9.9. Estructura Completa del Proyecto	74
9.10. Comandos Útiles	75
9.11. Resolución de Problemas	75
9.11.1. Error de Conexión a Base de Datos	75
9.11.2. Puerto Ocupado	76
9.11.3. Error de Compilación	76
9.11.4. Error de JWT	76
9.11.5. WebSocket no conecta	76

Índice de figuras

2.1. Pantalla principal del Dashboard de PetClinic	12
5.1. Pantalla de inicio de sesión con formulario y opción de agenda pública	32
5.2. Dashboard principal con 7 tarjetas de estadísticas	33
5.3. Agenda de citas con FullCalendar en vista mensual	34
5.4. Modal de creación/edición de cita	35
5.5. Módulos de clientes y mascotas	35
5.6. Módulo de facturación con items dinámicos	37
5.7. Reportes de ingresos con Chart.js	38
5.8. Odontograma canino interactivo con 42 dientes y 8 estados	39
5.9. Portal del cliente con citas, mascotas y facturas	40
6.1. Registro de auditoría con detalle de acciones por usuario	48
7.1. Gestión de veterinarios desde el panel de administración	51
7.2. Catálogo de servicios con precios y activación	53
7.3. Inventario de medicamentos con control de stock	54
7.4. Historial médico de mascota con notas de consulta	55
7.5. Registro de vacunas por mascota con alertas	56
7.6. Pacientes hospitalizados con control de ingresos	57
7.7. Plan de tratamiento con pasos secuenciados y avance	58
7.8. Consentimiento informado con firma digital y PDF	59
7.9. Log de auditoría con acciones del sistema	61
7.10. Panel de configuración general del sistema	62

Listings

2.1. Configuración WebSocket	15
3.1. Entidad Veterinario	18
3.2. Entidad Cliente	19
3.3. Entidad Mascota	19
3.4. Entidad Cita	19
3.5. Entidad Factura	20
4.1. Método JPQL para detectar solapamiento	26
4.2. Generación de PDF de factura	27
4.3. Creación de Payment Intent en PagoOnlineService	27
4.4. Configuración principal	28
5.1. Inicialización de FullCalendar	33
5.2. Creación de factura con items dinámicos	36
5.3. Renderizado de gráfico de ingresos con Chart.js	37
5.4. Cliente HTTP con autenticación JWT	41
6.1. JwtTokenProvider: generación y validación de tokens	43
6.2. Filtro de autenticación JWT	44
6.3. Configuración de seguridad con reglas de acceso	46
6.4. CustomUserDetailsService	46
6.5. Ejemplo de registro de auditoría	47
6.6. Configuración CORS	49
7.1. Programación de alertas de vacunación	62
8.1. Comando para ejecutar todas las pruebas	64
8.2. Ejecutar prueba individual	64
8.3. Ejemplo: autenticación como admin y prueba de dashboard	65
8.4. Configuración para pruebas de integración	67
8.5. Prueba de integración para creación de factura	67
8.6. Prueba de agenda pública sin autenticación	68
9.1. Creación de base de datos y usuario en PostgreSQL	70
9.2. Verificar que la base de datos es accesible	70
9.3. Ejecutar la aplicación en modo desarrollo	71
9.4. Compilar y empaquetar como JAR	71
9.5. Ejecutar JAR compilado con variables de entorno	71
9.6. Dockerfile multi-etapa para imagen optimizada	71

9.7. Docker Compose con PostgreSQL y aplicación	72
9.8. Construir y ejecutar con Docker Compose	72
9.9. Archivo de servicio systemd	72
9.10. Instalar y gestionar el servicio	73
9.11. Configuración de Nginx como proxy inverso	73
9.12. Verificar conexión PostgreSQL	75

Capítulo 1

Introducción

1.1 Descripción del Proyecto

PetClinic es un sistema de gestión para clínicas veterinarias desarrollado como aplicación web moderna. El sistema permite administrar todos los aspectos operativos de una clínica veterinaria: gestión de clientes, pacientes (mascotas), citas, facturación, inventario de medicamentos, historial médico, hospitalización, planes de tratamiento, consentimientos informados, odontograma canino, y más.

El proyecto está construido con una arquitectura de capas utilizando Java 21 y Spring Boot 3.2.3 en el backend, con una base de datos PostgreSQL, y un frontend SPA (*Single-Page Application*) construido con Bootstrap 5, FullCalendar 6 y Chart.js.

1.1.1 Objetivos del Sistema

- Digitalizar y centralizar la información de pacientes y clientes.
- Facilitar la gestión de citas con un calendario interactivo.
- Automatizar la facturación con generación de PDF y control de inventario.
- Proveer un portal de cliente para consulta de citas y facturas.
- Implementar alertas automáticas de vacunación vía correo electrónico.
- Gestionar hospitalizaciones, planes de tratamiento y consentimientos.
- Incluir un odontograma canino interactivo para diagnóstico dental.
- Integrar pagos en línea mediante Stripe.

1.1.2 Público Objetivo

El sistema está diseñado para:

- **Administradores:** Control total del sistema, gestión de usuarios y auditoría.
- **Veterinarios:** Gestión clínica completa (citas, historial, facturación).
- **Recepcionistas:** Lectura de datos y creación de clientes/citas.
- **Clientes:** Portal propio para consultar citas, facturas e historial de mascotas.

1.2 Tecnologías Utilizadas

1.2.1 Lenguaje y Framework

- **Java 21:** Última versión LTS con características modernas como *Pattern Matching*, *Records*, y *Sealed Classes*.
- **Spring Boot 3.2.3:** Framework de aplicaciones que simplifica la configuración y el despliegue.
- **Spring Data JPA / Hibernate:** ORM para la capa de persistencia.
- **Spring Security:** Framework de autenticación y autorización.
- **Spring WebSocket:** Comunicación en tiempo real para notificaciones de citas.
- **Spring Mail:** Envío de correos electrónicos (alertas de vacunas, recordatorios).
- **Spring AOP:** Programación orientada a aspectos.

1.2.2 Base de Datos

- **PostgreSQL 14+:** Base de datos relacional en producción.
- **H2:** Base de datos en memoria para pruebas.

1.2.3 Seguridad

- **JWT 0.12.3:** Biblioteca para generación y validación de tokens JWT.
- **BCrypt:** Algoritmo de hashing para contraseñas.

1.2.4 Frontend

- **Bootstrap 5.3:** Framework CSS para diseño responsive.
- **Bootstrap Icons:** Conjunto de iconos vectoriales.
- **FullCalendar 6.1:** Calendario interactivo para gestión de citas.
- **Chart.js 4.4:** Biblioteca de gráficos para reportes.
- **Vanilla JS:** JavaScript nativo sin frameworks adicionales, con patrón de módulos por sección.

1.2.5 Otras Tecnologías

- **iText 7 Core 8.0.3:** Generación de PDFs (facturas, consentimientos).
- **Apache POI 5.2.5:** Generación de archivos Excel (reportes de ingresos).
- **Stripe Java 28.1.0:** Integración de pagos en línea.
- **Maven:** Herramienta de construcción y gestión de dependencias.
- **JUnit 5:** Framework de pruebas unitarias y de integración.

1.3 Convenciones del Proyecto

- **Arquitectura en capas:** Controller → Service → Repository → DB.
- **DTO Pattern:** Separación de entidades JPA de los objetos de transferencia de datos.
- **Inyección de dependencias:** Constructor injection en todas las clases.
- **Global Exception Handler:** Manejo centralizado de excepciones con `@RestControllerAdvice`.
- **Auditoría:** Registro de acciones CRUD con detalles de usuario y dirección IP.
- **WebSocket:** Notificaciones en tiempo real para cambios en citas.

1.4 Requisitos del Sistema

- Java 21 o superior.
- PostgreSQL 14 o superior.
- Maven 3.8+.

- Navegador web moderno (Chrome, Firefox, Edge).
- (Opcional) Cuenta de Stripe para pagos en línea.
- (Opcional) Cuenta SMTP para envío de correos.

Capítulo 2

Arquitectura del Sistema

2.1 Visión General

PetClinic sigue una arquitectura en capas (*Layered Architecture*) con una SPA (*Single Page Application*) en el frontend que se comunica con el backend mediante una API REST protegida con JWT. El backend sigue el patrón Controller → Service → Repository → DB, con una separación estricta de responsabilidades.

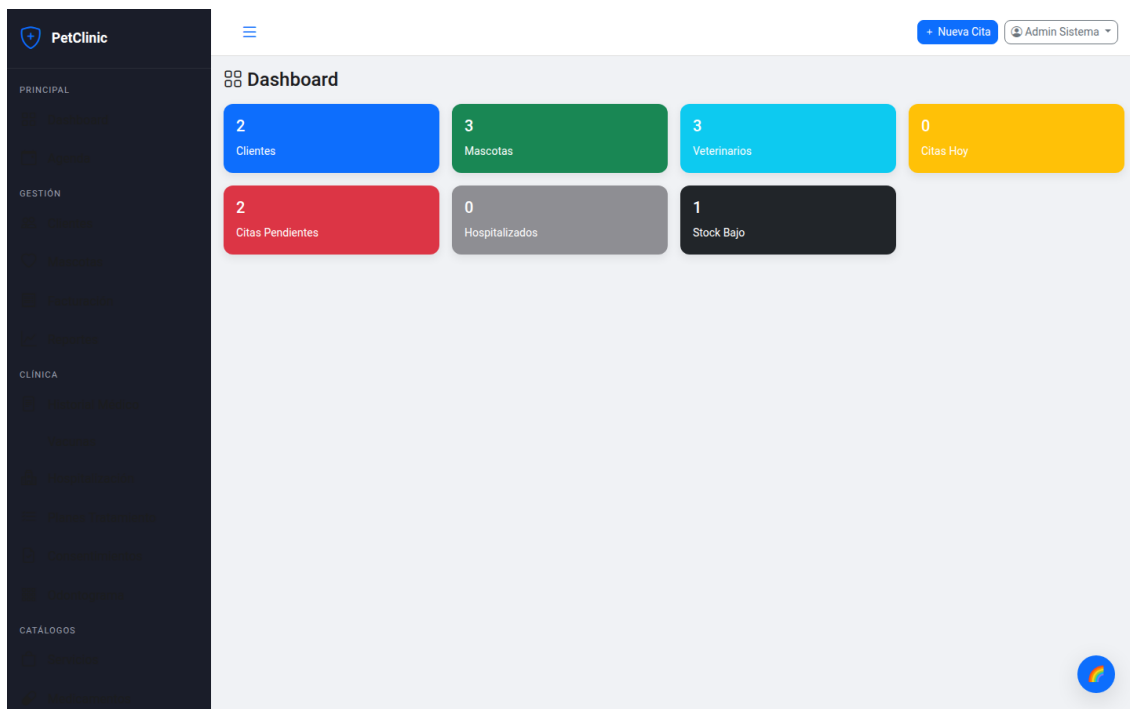
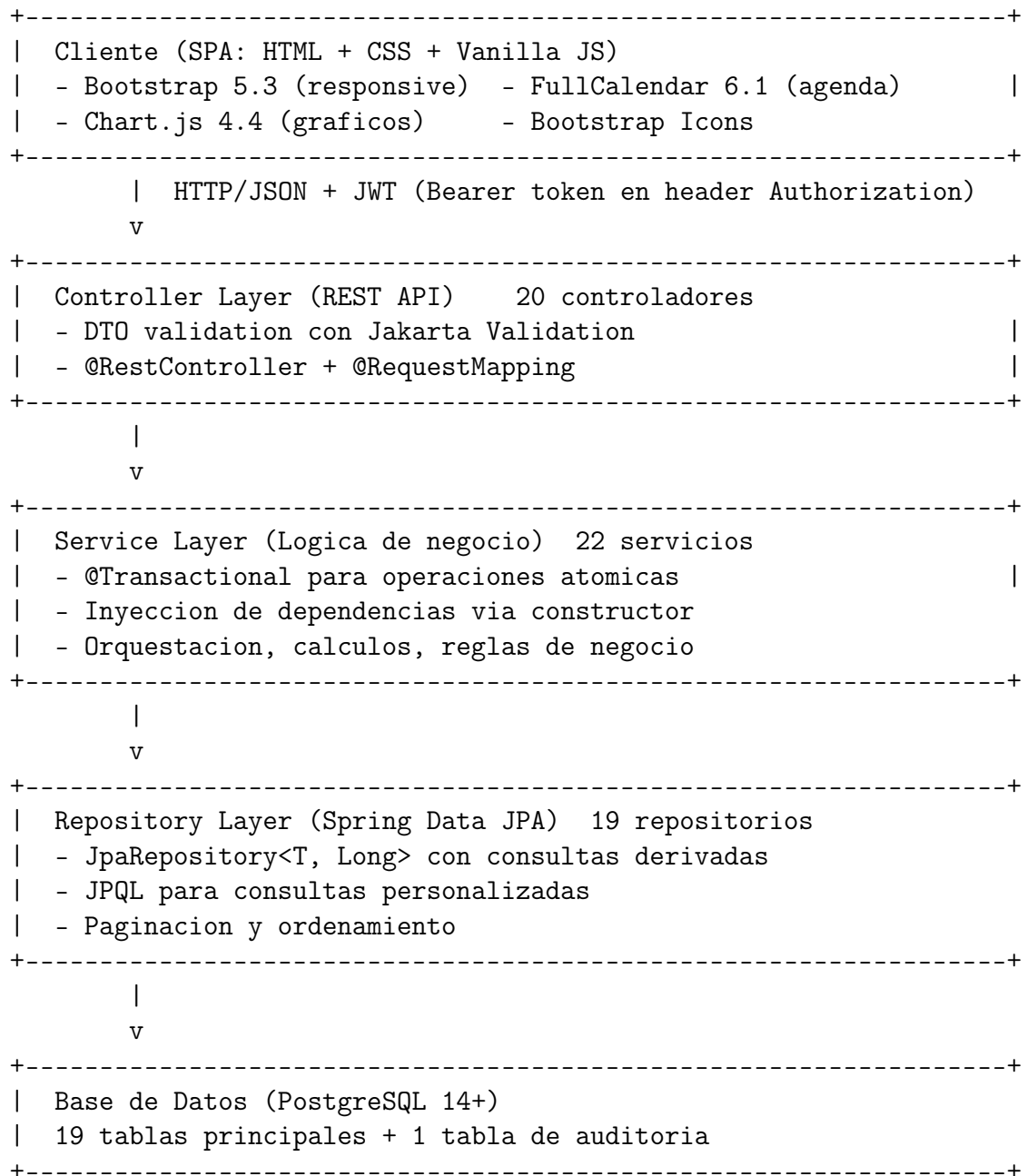


Figura 2.1: Pantalla principal del Dashboard de PetClinic

2.1.1 Diagrama de Capas



2.2 Flujo de una Petición

Cada petición HTTP atraviesa los siguientes componentes en orden:

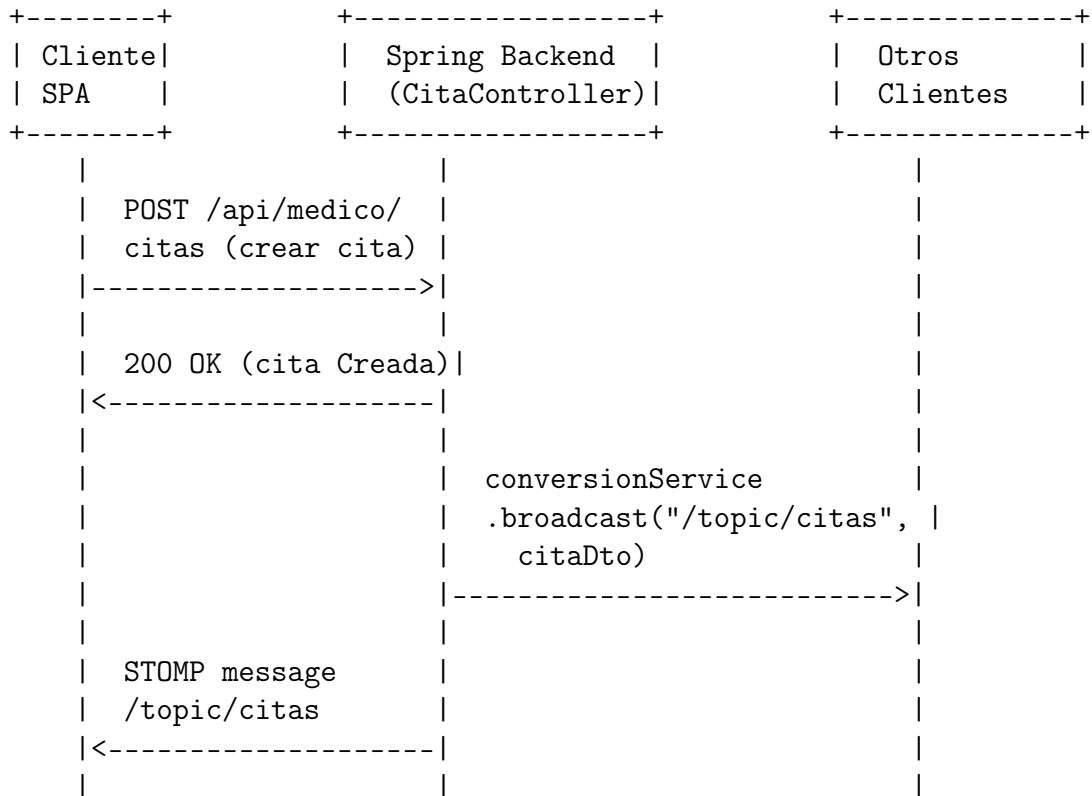
1. **JwtAuthFilter:** Extrae el token del header **Authorization: Bearer <token>**, valida la firma HMAC-SHA384, verifica expiración y extrae los claims (email, rol, userId). Si

es válido, establece la autenticación en el `SecurityContextHolder`.

2. **SecurityConfig:** Verifica que el rol del usuario (ADMIN, VETERINARIO, RECEPTIONISTA, CLIENTE) tenga permiso para el endpoint solicitado mediante reglas `authorizeHttpRequests`.
3. **Controller:** Recibe la petición HTTP, valida los DTOs con `@Valid` y `jakarta.validation`, y delega la lógica al servicio correspondiente.
4. **Service:** Ejecuta la lógica de negocio dentro de una transacción (`@Transactional`). Realiza validaciones, cálculos y orquestación de operaciones.
5. **Repository:** Interactúa con la base de datos mediante Spring Data JPA. Las consultas personalizadas se definen con JPQL.
6. **GlobalExceptionHandler:** Captura cualquier excepción no manejada y la formatea como una respuesta JSON estándar con código HTTP apropiado.

2.3 Flujo de Notificaciones en Tiempo Real (WebSocket)

PetClinic utiliza WebSocket con STOMP sobre SockJS para notificaciones en tiempo real de cambios en citas.



La configuración WebSocket se implementa de la siguiente manera:

```

1 @Configuration
2 @EnableWebSocketMessageBroker
3 public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
4     @Override
5     public void configureMessageBroker(MessageBrokerRegistry config) {
6         config.enableSimpleBroker("/topic");
7         config.setApplicationDestinationPrefixes("/app");
8     }
9
10    @Override
11    public void registerStompEndpoints(StompEndpointRegistry registry) {
12        registry.addEndpoint("/ws")
13            .setAllowedOriginPatterns("*")
14            .withSockJS();
15    }
16 }

```

Listing 2.1: Configuración WebSocket

2.4 Patrones de Diseño

2.4.1 Patrones Implementados

Patrón	Implementación
Singleton	Spring Beans (@Service, @Repository, @Controller)
Inyección de Dependencias	Constructor injection en todas las clases
Factory	Spring Data JPA genera repositorios automáticamente
Chain of Responsibility	Security Filter Chain (JwtAuthFilter → UsernamePasswordAuthenticationFilter)
Observer / Pub-Sub	WebSocket STOMP para actualizaciones de citas en tiempo real
Strategy	PasswordEncoder (BCrypt) intercambiable
Template Method	JPA Lifecycle Callbacks (@PrePersist, @PreUpdate)
DTO Pattern	CitaDto, FacturaDto, LoginRequest/Response separan API de entidades
Global Exception Handler	@RestControllerAdvice con GlobalExceptionHandler
CommandLineRunner	DataInitializer para seed data al arrancar
DAO Pattern	Repositorios encapsulan operaciones de base de datos
Scheduling	@Scheduled en VacunaScheduler para alertas automáticas

Facade	Servicios actúan como fachada ante operaciones complejas (ej: <code>FacturaService</code>)
--------	---

2.5 Diagrama de Componentes

2.5.1 Capas del Sistema

Capa de Presentación (Frontend) SPA construida con HTML, CSS y JavaScript vanilla. Utiliza Bootstrap 5 para el diseño responsive, FullCalendar 6 para la agenda de citas, Chart.js 4 para gráficos de reportes. Se comunica con el backend mediante peticiones HTTP REST con autenticación JWT. Las actualizaciones en tiempo real llegan vía WebSocket STOMP.

Capa de API REST (Controller) 20 controladores REST que exponen endpoints para cada módulo del sistema. Los controladores reciben y devuelven DTOs, nunca entidades JPA directamente. Utilizan `@Valid` para validación automática de entrada y `ResponseEntity` para control de códigos HTTP.

Capa de Negocio (Service) 22 servicios que encapsulan toda la lógica de negocio. Los servicios son transaccionales y contienen validaciones, cálculos y orquestación de operaciones. Ejemplos: `CitaService` valida disponibilidad, `FacturaService` maneja descuento de stock, `PdfService` genera documentos.

Capa de Persistencia (Repository) 19 interfaces de repositorio que extienden `JpaRepository` con consultas personalizadas mediante JPQL y métodos derivados. Cada repositorio incluye consultas específicas como búsqueda por texto libre, agregaciones y filtros por estado.

Capa de Seguridad 4 componentes de seguridad: `JwtTokenProvider` genera y valida tokens, `JwtAuthFilter` intercepta peticiones, `SecurityConfig` configura reglas de acceso, `CustomUserDetailsService` carga usuarios de la base de datos.

2.6 Esquema de Navegación

La aplicación cuenta con dos modos de navegación:

- **Modo Staff (Personal):** Barra lateral con secciones para administración clínica (Dashboard, Agenda, Clientes, Mascotas, Facturación, Reportes, Historial Médico, Vacunas, Hospitalización, Planes, Consentimientos, Odontograma, Servicios, Medicamentos, Admin, Auditoría, Configuración).

- **Modo Cliente:** Barra lateral simplificada con secciones personales (Inicio, Mis Mascotas, Mis Citas, Mis Facturas, Mi Perfil).

Capítulo 3

Modelo de Datos

3.1 Diagrama Entidad-Relación

El sistema cuenta con 19 tablas principales más 1 tabla de auditoría. JPA genera automáticamente el esquema con `spring.jpa.hibernate.ddl-auto=update`.

3.1.1 Entidades del Sistema

Veterinario (veterinarios)

Almacena los usuarios del sistema: administradores, veterinarios y recepcionistas.

```
1 @Entity @Table(name = "veterinarios")
2 public class Veterinario {
3     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
4     private Long id;
5     @NotBlank private String nombre;
6     @NotBlank private String apellido;
7     @NotBlank @Column(unique = true) private String email;
8     @NotBlank private String passwordHash; // BCrypt
9     private String telefono;
10    private String especialidad;
11    private String cedulaProfesional;
12    @Enumerated(EnumType.STRING) private RolUsuario rol;
13    private LocalTime horarioInicio;
14    private LocalTime horarioFin;
15    private Integer duracionTurnoMinutos = 30;
16    private Boolean activo = true;
17    // Auditoria: createdAt, updatedAt
18 }
```

Listing 3.1: Entidad Veterinario

Cliente (clientes)

Dueños de mascotas con opción de portal web.

```

1 @Entity @Table(name = "clientes")
2 public class Cliente {
3     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
4     private Long id;
5     @NotBlank private String nombre;
6     @NotBlank private String apellido;
7     @Email @NotBlank private String email;
8     private String telefono;
9     private String direccion;
10    private String portalEmail;           // Login para portal cliente
11    private String portalPasswordHash;    // BCrypt
12    private Boolean portalActivo = false;
13    @OneToMany(mappedBy = "cliente") private List<Mascota> mascotas;
14 }

```

Listing 3.2: Entidad Cliente

Mascota (mascotas)

Pacientes del sistema, vinculados a un cliente dueño.

```

1 @Entity @Table(name = "mascotas")
2 public class Mascota {
3     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
4     private Long id;
5     @NotBlank private String nombre;
6     @Enumerated(EnumType.STRING) private Especie especie;
7     private String raza, color;
8     @Enumerated(EnumType.STRING) private GeneroMascota genero;
9     private LocalDate fechaNacimiento;
10    private Double peso;
11    @Column(columnDefinition = "TEXT") private String alergias;
12    @Column(columnDefinition = "TEXT") private String condicionesMedicas;
13    @ManyToOne @JoinColumn(name = "cliente_id") private Cliente cliente;
14 }

```

Listing 3.3: Entidad Mascota

Cita (citas)

Agendamiento de consultas con control de disponibilidad.

```

1 @Entity @Table(name = "citas")
2 public class Cita {
3     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
4     private Long id;
5     @ManyToOne private Mascota mascota;
6     @ManyToOne private Veterinario veterinario;

```

```

7  @ManyToOne private Cliente cliente;
8  @NotNull private LocalDateTime fechaHoraInicio;
9  @NotNull private LocalDateTime fechaHoraFin;
10 @NotBlank private String motivo;
11 @Column(columnDefinition = "TEXT") private String notas;
12 @Enumerated(EnumType.STRING) private EstadoCita estado;
13 private String motivoCancelacion;
14 }

```

Listing 3.4: Entidad Cita

Factura (facturas)

Facturación con detalle de items y control de pagos.

```

1  @Entity @Table(name = "facturas")
2  public class Factura {
3      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
4      private Long id;
5      @NotBlank @Column(unique = true) private String numeroFactura;
6      @ManyToOne private Cliente cliente;
7      @ManyToOne private Veterinario veterinario;
8      @ManyToOne private Cita cita; // Opcional
9      private LocalDateTime fechaEmision;
10     private Double subtotal = 0.0;
11     private Double descuentoTotal = 0.0;
12     private Double total = 0.0;
13     @Enumerated(EnumType.STRING) private EstadoFactura estado;
14     private Double totalPagado = 0.0;
15     @OneToMany(mappedBy = "factura", cascade = CascadeType.ALL)
16     private List<DetalleFactura> detalles;
17     @OneToMany(mappedBy = "factura", cascade = CascadeType.ALL)
18     private List<Pago> pagos;
19 }

```

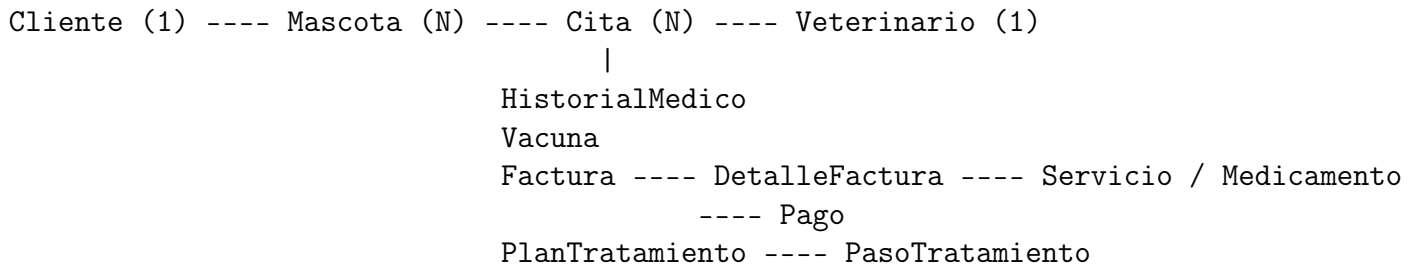
Listing 3.5: Entidad Factura

3.1.2 Enumeraciones

Enumeración	Valores
RolUsuario	ADMIN, VETERINARIO, RECEPCIONISTA
EstadoCita	PROGRAMADA, CONFIRMADA, EN_CURSO, COMPLETADA, CANCELADA, NO_ASISTIO
EstadoFactura	PENDIENTE, PAGADA_PARCIAL, PAGADA, ANULADA
MetodoPago	EFFECTIVO, TARJETA_CREDITO, TARJETA_DEBITO, TRANSFERENCIA, CHEQUE

Especie	CANINO, FELINO, EQUINO, BOVINO, AVIAR, EXOTICO, OTRO
GeneroMascota	MACHO, HEMBRA
EstadoHospitalizacion	HOSPITALIZADO, DADO_ALTA
EstadoPlan	ACTIVO, COMPLETADO, CANCELADO
EstadoPaso	PENDIENTE, EN_PROCESO, COMPLETADO, OMITIDO
TipoMovimiento	ENTRADA, SALIDA, AJUSTE

3.1.3 Relaciones Principales



3.1.4 Listado Completo de Repositorios

Repositorio	Consultas Personalizadas
ClienteRepository	findByPortalEmail, findByEmail, búsqueda de texto libre (nombre, apellido, email)
MascotaRepository	findByClienteId, findByClienteIdOrderByNombreAsc
VeterinarioRepository	findByEmail, findByRol, findByActivoTrue, existsByEmail
CitaRepository	findByVeterinarioId, findByClienteId, findByMascotaId, findByEstado, findCitasOcupadas (JPQL), findCitasDelDia (JPQL)
ServicioRepository	findByActivoTrue
MedicamentoRepository	findByActivoTrue, findByStockActualLessThanEqual
FacturaRepository	findByClienteId, sumIngresosByFecha (JPQL aggregate)
VacunaRepository	findByMascotaId, findByFechaProximaDosisBetween
HistorialMedicoRepository	findByMascotaId, findByCitaId
HospitalizacionRepository	findByMascotaId, findByEstado
PlanTratamientoRepository	findByMascotaId
PasoTratamientoRepository	findByPlanTratamientoIdOrderByOrdenAsc
ConsentimientoRepository	findByClienteId, findByVeterinarioId
OdontogramaRepository	findByMascotaId
MovimientoInventarioRepository	findByMedicamentoId

AuditLogRepository	findAllByOrderByFechaAccionDesc
--------------------	---------------------------------

Capítulo 4

Backend

4.1 Estructura del Proyecto

El backend sigue la estructura estándar de Maven con Spring Boot:

```
src/main/java/com/veterinary/
|-- VeterinaryApplication.java      # Entry point @SpringBootApplication
|-- config/                        # Configuraciones (4)
|   |-- WebConfig.java             # Configuracion web y CORS
|   |-- WebSocketConfig.java       # STOMP over SockJS
|   |-- MailConfig.java            # JavaMailSender
|   |-- JacksonConfig.java         # ObjectMapper personalizado
|-- controller/                   # REST Controllers (20)
|-- domain/                       # Entidades JPA (20 + 9 enums)
|-- dto/                          # Data Transfer Objects (6)
|-- repository/                   # Spring Data JPA (19)
|-- scheduler/                    # Tareas programadas (1)
|   |-- VacunaScheduler.java        # Alertas de vacunacion
|-- security/                     # Seguridad JWT (4)
|   |-- JwtTokenProvider.java       # Generacion/validacion JWT
|   |-- JwtAuthFilter.java          # Filtro de autenticacion
|   |-- SecurityConfig.java         # Reglas de acceso
|   |-- CustomUserDetailsService.java # Carga de usuarios
|-- service/                      # Servicios (22)
|   |-- CitaService.java            # Gestion de citas
|   |-- FacturaService.java         # Facturacion y pagos
|   |-- PdfService.java             # Generacion de PDFs iText 7
|   |-- ReporteService.java         # Reportes y exportacion Excel
|   |-- NotificacionService.java    # Emails con plantillas HTML
|   |-- PagoOnlineService.java      # Integracion Stripe
|   |-- AuditService.java           # Registro de auditoria
```



```
|-- ... (15 servicios mas)
```

4.2 Controladores REST

El sistema expone 20 controladores REST que cubren todos los módulos:

4.2.1 Autenticación

`AuthController` – `/api/auth`

- `POST /login`: Autentica personal o cliente, retorna JWT con claims (email, rol, userId).

4.2.2 Gestión Clínica

Todos los siguientes endpoints están bajo `/api/medico/` y requieren autenticación JWT:

`DashboardController`

- `GET /dashboard`: Estadísticas del sistema (clientes, mascotas, citas hoy, ingresos del mes, stock bajo).

`VeterinarioController`

- CRUD completo de veterinarios (sólo ADMIN).
- `PUT /perfil`: Actualizar perfil propio.
- `POST /cambiar-password`: Cambiar contraseña con verificación de contraseña actual.

`ClienteController`

- CRUD de clientes con búsqueda de texto libre (`?q=texto`) sobre nombre, apellido y email.

`MascotaController`

- CRUD de mascotas con listado por cliente.
- Vista detallada con citas, historial y vacunas.

`CitaController`

- CRUD de citas con validación de solapamiento.
- WebSocket broadcast en `/topic/citas` al crear/actualizar/eliminar.
- Notificación por email al crear la cita.

ServicioController – Catálogo de servicios con activación/desactivación. **MedicamentoController** – Inventario con ajuste de stock y alertas de stock bajo. **VacunaController** – Registro de vacunas por mascota con control de próxima dosis. **HistorialMedicoController** – Notas de consulta vinculadas a citas. **HospitalizacionController** – Ingreso/alta de mascotas con control de jaula. **PlanTratamientoController** – Planes con pasos secuenciados y estados. **ConsentimientoController** – Consentimientos con firma digital y generación de PDF. **OdontogramaController** – Diagrama dental con 42 dientes y 8 estados. **FacturaController** – Facturación con PDF, pagos parciales y control de saldo. **ReporteController** – Reporte de ingresos por período con exportación a Excel.

4.2.3 Portal Cliente

PortalClienteController – /api/portal

- GET /perfil: Perfil del cliente autenticado.
- GET /citas: Citas del cliente.
- GET /facturas: Facturas del cliente.
- GET /mascotas: Mascotas del cliente con historial.

4.2.4 Público

PublicBookingController – /api/public (sin autenticación)

- GET /veterinarios: Lista de veterinarios activos con horarios.
- GET /slots?vetId=&fecha=: Slots disponibles calculados por horario y citas existentes.
- POST /citas: Crear cita como visitante (validación de email contra BD).

4.2.5 Pagos Online

PagoOnlineController – /api/pagos

- POST /stripe/create-payment-intent: Crea intención de pago Stripe vinculada a una factura.
- POST /stripe/webhook: Webhook de confirmación de pago.

4.2.6 Auditoría

AuditController – /api/admin/auditoria

- GET /: Listado paginado de logs de auditoría ordenados por fecha descendente.

4.3 Capa de Servicios

22 servicios que encapsulan la lógica de negocio. Los más relevantes:

- **CitaService**: Validación de disponibilidad (evita solapamientos usando JPQL), notificaciones por email y broadcast WebSocket.
- **FacturaService**: Creación automática de número de factura (FAC-yyyyMMdd-HHmms-XXXX), descuento de stock al incluir medicamentos, control de pagos parciales.
- **MedicamentoService**: Ajuste de stock con registro automático de movimiento de inventario (trazabilidad completa).
- **NotificacionService**: Plantillas HTML para emails de recordatorios, alertas de vacunas y notificaciones de facturas.
- **PdfService**: Generación de PDFs con iText 7 (facturas con detalle de items, consentimientos informados).
- **ReporteService**: Reporte de ingresos con Chart.js y exportación a Excel con Apache POI.
- **PagoOnlineService**: Integración con Stripe Payment Intents en modo sandbox cuando no hay API key configurada.
- **PublicBookingService**: Cálculo de slots disponibles basado en horarios de veterinarios y citas existentes.
- **AuditService**: Registro centralizado de operaciones CRUD con metadatos (usuario, IP, fecha).

4.3.1 Ejemplo: Validación de Solapamiento de Citas

```
1 @Query("SELECT c FROM Cita c WHERE c.veterinario.id = :vetId "
2       + "AND c.estado NOT IN ('CANCELADA', 'NO_ASISTIO') "
3       + "AND c.fechaHoraInicio < :fin AND c.fechaHoraFin > :inicio "
4       + "AND (:citaExcluirId IS NULL OR c.id <> :citaExcluirId)")
5 List<Cita> findCitasOcupadas(@Param("vetId") Long vetId,
6                             @Param("inicio") LocalDateTime inicio,
7                             @Param("fin") LocalDateTime fin,
8                             @Param("citaExcluirId") Long citaExcluirId);
```

Listing 4.1: Método JPQL para detectar solapamiento

4.3.2 Ejemplo: Generación de PDF con iText 7

```

1 public ByteArrayInputStream generarFacturaPdf(Factura factura) {
2     ByteArrayOutputStream out = new ByteArrayOutputStream();
3     PdfWriter writer = new PdfWriter(out);
4     PdfDocument pdf = new PdfDocument(writer);
5     Document document = new Document(pdf, PageSize.A4);
6
7     document.add(new Paragraph("PETCLINIC VETERINARY CLINIC")
8         .setFontSize(20).setBold().setTextAlignment(TextAlignment.CENTER))
9     ;
10    document.add(new Paragraph("Factura: " + factura.getNumeroFactura())
11        .setFontSize(14).setTextAlignment(TextAlignment.CENTER));
12    document.add(new Paragraph(" "));
13
14    document.add(new Paragraph("Cliente: "
15        + factura.getCliente().getNombre() + " "
16        + factura.getCliente().getApellido()));
17    document.add(new Paragraph("Fecha: "
18        + factura.getFechaEmision().format(
19            DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm"))));
20
21    Table table = new Table(new float[]{3, 1, 1, 1});
22    table.addCell("Descripcion");
23    table.addCell("Cant.");
24    table.addCell("P. Unit.");
25    table.addCell("Subtotal");
26    for (DetalleFactura d : factura.getDetalles()) {
27        table.addCell(d.getDescripcionLinea());
28        table.addCell(String.valueOf(d.getCantidad()));
29        table.addCell("$" + d.getPrecioUnitario());
30        table.addCell("$" + d.getSubtotal());
31    }
32    document.add(table);
33    document.add(new Paragraph("Total: $" + factura.getTotal())
34        .setFontSize(16).setBold().setTextAlignment(TextAlignment.RIGHT));
35    document.close();
36    return new ByteArrayInputStream(out.toByteArray());
37 }

```

Listing 4.2: Generación de PDF de factura

4.3.3 Ejemplo: Integración con Stripe

```

1 public String createPaymentIntent(Long facturaId) {
2     Factura factura = facturaRepo.findById(facturaId)
3         .orElseThrow(() -> new RuntimeException("Factura no encontrada"));
4
5     // Stripe maneja montos en centavos
6     long montoCentavos = (long)(factura.getTotal() * 100);

```

```
7
8     PaymentIntentCreateParams params = PaymentIntentCreateParams.builder()
9         .setAmount(montoCentavos)
10        .setCurrency("mxn")
11        .putMetadata("facturaId", facturaId.toString())
12        .putMetadata("clienteId", factura.getCliente().getId().toString())
13        .build();
14
15    try {
16        PaymentIntent intent = PaymentIntent.create(params);
17        factura.setStripePaymentIntentId(intent.getId());
18        facturaRepo.save(factura);
19        return intent.getClientSecret();
20    } catch (StripeException e) {
21        throw new RuntimeException("Error al crear payment intent: "
22            + e.getMessage());
23    }
24 }
```

Listing 4.3: Creación de Payment Intent en PagoOnlineService

4.4 Configuración

4.4.1 Application Properties

```
1 server.port=8090
2
3 # Base de Datos PostgreSQL
4 spring.datasource.url=jdbc:postgresql://localhost:5432/veterinary_db
5 spring.datasource.username=veterinary_user
6 spring.datasource.password=veterinary_pass
7
8 # JPA / Hibernate
9 spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
    PostgreSQLDialect
11 spring.jpa.show-sql=false
12
13 # JWT
14 app.jwt.secret=${JWT_SECRET:ClaveSuperSeguraParaFirmarTokensJWT2024}
15 app.jwt.expiration=${JWT_EXPIRATION:1800000}
16
17 # Stripe
18 stripe.api-key=${STRIPE_API_KEY:sk_test_placeholder}
19 stripe.webhook-secret=${STRIPE_WEBHOOK_SECRET:whsec_placeholder}
20
21 # Mail (SMTP)
22 spring.mail.host=${MAIL_HOST:smtp.gmail.com}
23 spring.mail.port=${MAIL_PORT:587}
```

```

24 spring.mail.username=${MAIL_USERNAME:}
25 spring.mail.password=${MAIL_PASSWORD:}
26 spring.mail.properties.mail.smtp.auth=true
27 spring.mail.properties.mail.smtp.starttls.enable=true
28
29 # Clinica
30 app.clinic.nombre=PetClinic Veterinary Clinic
31 app.clinic.email=contacto@petclinic.com

```

Listing 4.4: Configuración principal

4.4.2 DataInitializer

Al arrancar por primera vez, `DataInitializer` inserta datos de prueba utilizando el patrón `CommandLineRunner`:

- 3 veterinarios: Admin Sistema (ADMIN, admin@petclinic.com/Admin123), Laura García (VETERINARIO, Cirugía, vet@petclinic.com/Vet123), Carlos López (RECEPCIONISTA, recepcion@petclinic.com/Recep123).
- 2 clientes: Ana Martínez (ana@email.com, con portal activo/Ana123), Pedro Ramírez (pedro@email.com).
- 3 mascotas: Max (Golden Retriever, 30kg), Luna (Siamés, 4kg), Rocky (Bulldog Francés, 12kg).
- 5 servicios: Consulta General (\$500), Vacunación (\$350), Cirugía (\$2,500), Desparasitación (\$200), Grooming (\$400).
- 3 medicamentos: Amoxicilina 500mg (100 tabs, stock mínimo 10), Meloxicam 5mg (50 ml, stock mínimo 5), Frontline Plus (3 pipetas, stock mínimo 2).
- 2 citas programadas para el día siguiente con estados PROGRAMADA y CONFIRMADA.

4.5 Manejo de Excepciones

El `GlobalExceptionHandler` captura y formatea errores de manera consistente retornando JSON con estructura `{error: mensaje, details: [...]}`:

Excepción	Código HTTP	Uso
<code>EntityNotFoundException</code>	404	Recurso no encontrado (ej: cliente, mascota, factura)

<code>IllegalArgumentException</code>	400	Parámetros inválidos (ej: fechas incorrectas, datos duplicados)
<code>AccessDeniedException</code>	403	Sin permisos para el recurso solicitado
<code>MethodArgumentNotValidException</code>	400	Errores de validación de DTOs (<code>@NotBlank</code> , <code>@Email</code> , etc.)
<code>RuntimeException</code>	Variable	Determinado por el mensaje (ej: “Stock insuficiente” → 400)
<code>Exception</code>	500	Error interno del servidor

Capítulo 5

Frontend

5.1 Arquitectura del Frontend

El frontend es una *Single Page Application* (SPA) construida con tecnología web estándar: HTML5, CSS3 y JavaScript vanilla. No utiliza frameworks de JavaScript como React, Angular o Vue, lo que facilita el despliegue y reduce dependencias. La aplicación utiliza un enrutador casero basado en el hash de la URL (`window.location.hash`) para la navegación entre secciones.

5.1.1 Estructura de Archivos

```
src/main/resources/static/
|-- index.html          # Pagina principal SPA (+600 lineas)
|-- css/
|   |-- app.css         # Estilos con 8 temas mediante variables CSS (+670 lineas)
|-- js/
|   |-- api.js          # Cliente HTTP con autenticacion JWT (+220 lineas)
|   |-- app.js          # Logica principal: routing, modulos, CRUDs (+1590 lineas)
|   |-- vet-extras.js   # Sistema de temas y utilidades (+110 lineas)
```

5.2 Pantalla de Login

La pantalla de inicio de sesión presenta un diseño de dos columnas con iconos de mascotas animados (CSS animations), un formulario de login con selección de tipo (Personal/Cliente) y una pestaña para agendar citas públicamente sin autenticación.

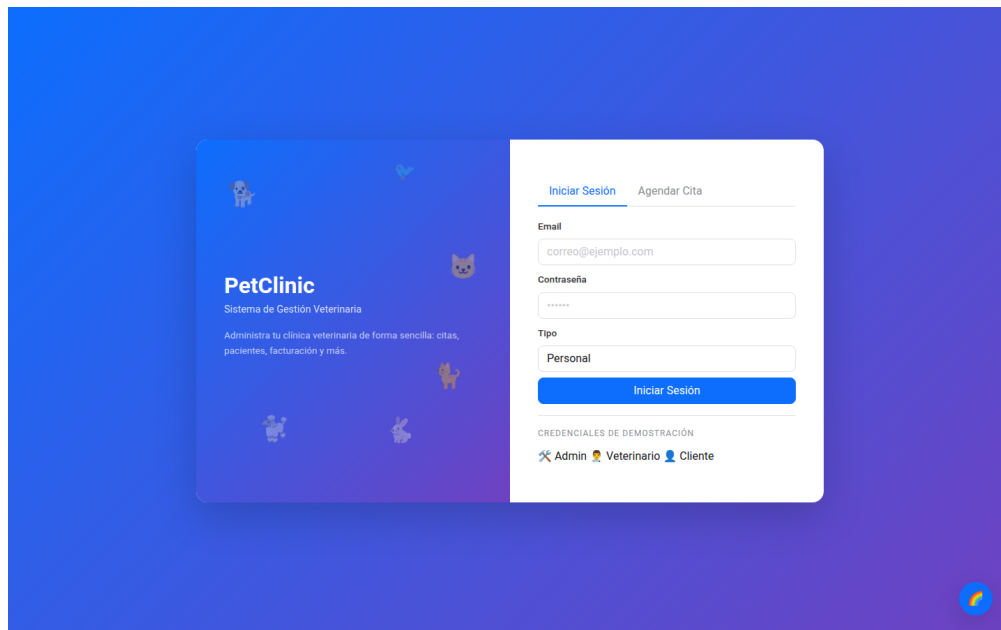


Figura 5.1: Pantalla de inicio de sesión con formulario y opción de agenda pública

5.3 Dashboard

El dashboard muestra 7 tarjetas con estadísticas clave del sistema en tiempo real: total de clientes, mascotas, veterinarios, citas hoy, citas pendientes, pacientes hospitalizados y medicamentos con stock bajo. Los datos se cargan mediante `apiGet('/api/medico/dashboard')` al entrar a la sección.

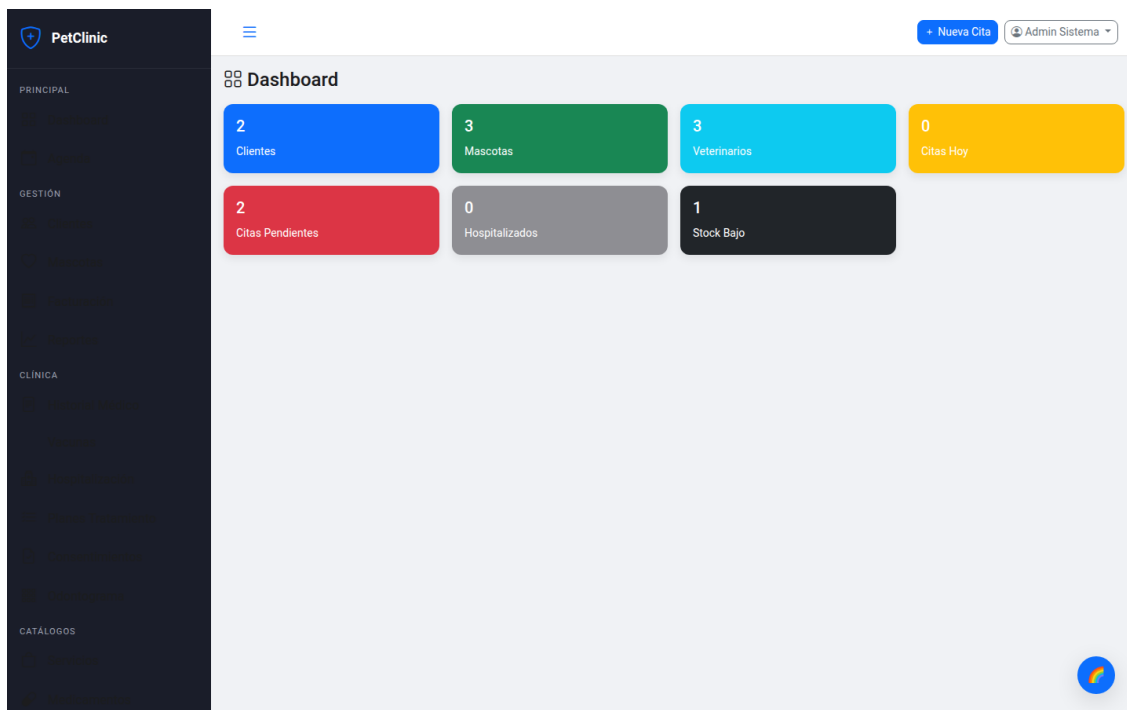


Figura 5.2: Dashboard principal con 7 tarjetas de estadísticas

5.4 Agenda de Citas

Utiliza FullCalendar 6 con vistas mensual, semanal y diaria. Soporta arrastrar y soltar para reprogramar citas, codificación por colores según el estado (verde=Completada, azul=Programada, amarillo=Confirmada, rojo=Cancelada), y actualizaciones en tiempo real mediante WebSocket.

```

1 const calendar = new FullCalendar.Calendar(calendarEl, {
2   plugins: [dayGridPlugin, timeGridPlugin, interactionPlugin],
3   headerToolbar: {
4     left: 'prev,next today',
5     center: 'title',
6     right: 'dayGridMonth,timeGridWeek,timeGridDay'
7   },
8   editable: true,
9   events: function(info, successCallback) {
10     apiGet('/api/medico/citas').then(data => {
11       successCallback(data.map(cita => ({
12         id: cita.id,
13         title: cita.mascotaNombre + ' - ' + cita.motivo,
14         start: cita.fechaHoraInicio,
15         end: cita.fechaHoraFin,
16         backgroundColor: coloresEstado[cita.estado],
17         extendedProps: { estado: cita.estado }
18       })));

```

```

19     });
20   },
21   eventDrop: function(info) {
22     apiPut('/api/medico/citas/' + info.event.id + '/reprogramar', {
23       fechaHoraInicio: info.event.start.toISOString(),
24       fechaHoraFin: info.event.end.toISOString()
25     });
26   }
27 });
28 calendar.render();

```

Listing 5.1: Inicialización de FullCalendar

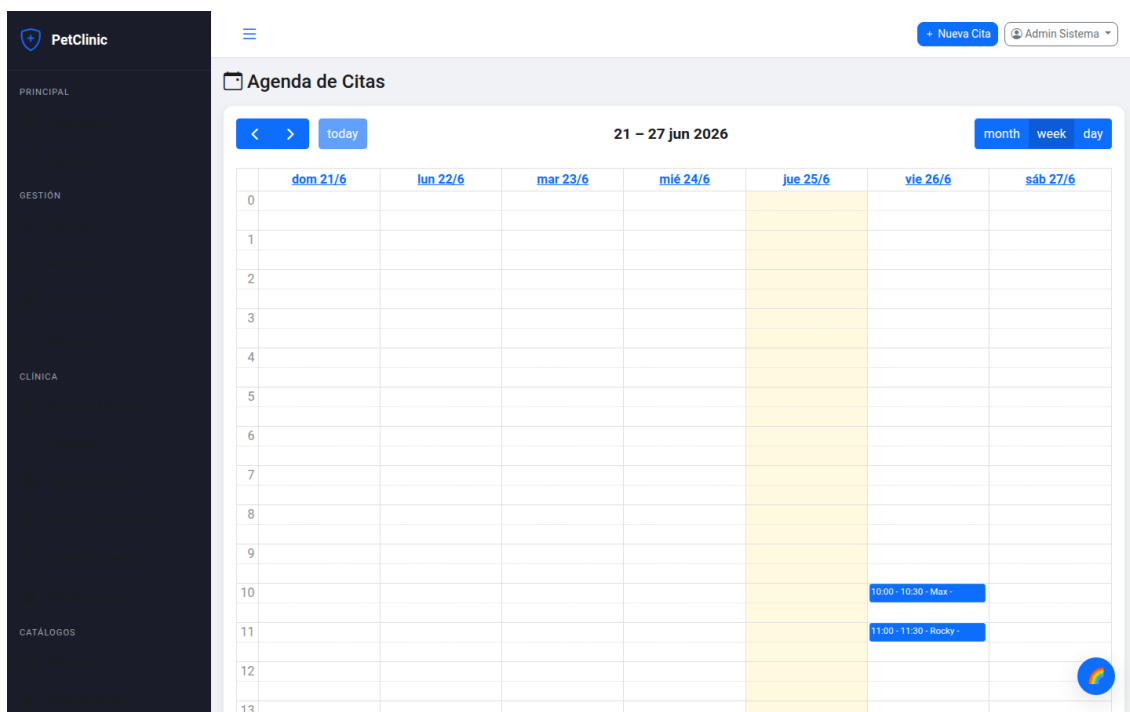


Figura 5.3: Agenda de citas con FullCalendar en vista mensual

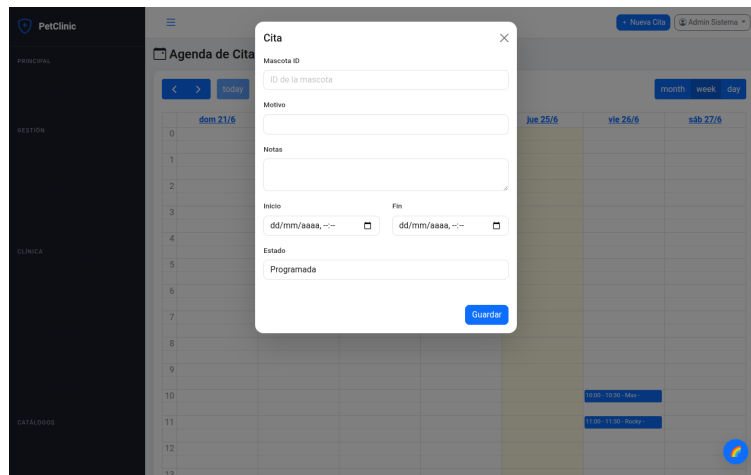
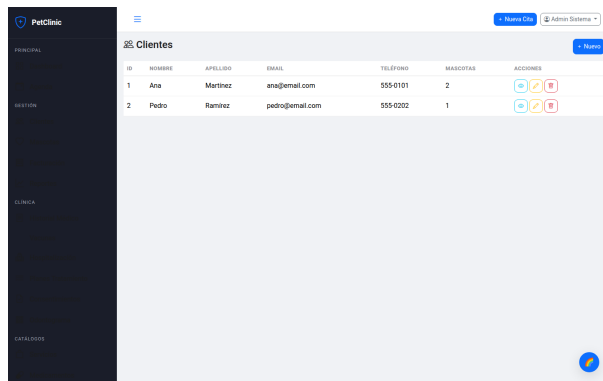


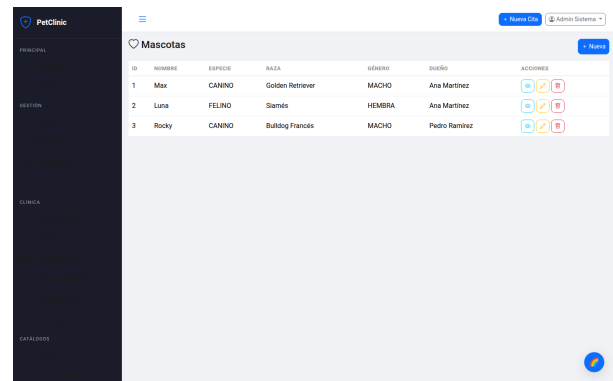
Figura 5.4: Modal de creación/edición de cita

5.5 Gestión de Clientes y Mascotas

El módulo de clientes presenta una tabla con búsqueda de texto libre, CRUD completo y vista detallada con mascotas asociadas. El módulo de mascotas incluye información detallada como especie, raza, peso, alergias y condiciones médicas. Ambos módulos utilizan modales Bootstrap para los formularios de creación/edición.



(a) Listado de clientes con búsqueda



(b) Listado de mascotas por cliente

Figura 5.5: Módulos de clientes y mascotas

5.6 Facturación

El módulo de facturación permite crear facturas con items dinámicos (servicios y medicamentos), cálculo automático de subtotales y totales, descuento automático de stock, registro de pagos parciales y generación de PDF con iText 7.

```
1 async function guardarFactura() {
2   const detalles = [];
3   document.querySelectorAll('#detalles-factura .item-row').forEach(row
4     => {
5     const tipoItem = row.querySelector('.tipo-item').value;
6     const servicioId = row.querySelector('.servicio-id')?.value;
7     const medicamentoId = row.querySelector('.medicamento-id')?.value;
8     const cantidad = parseInt(row.querySelector('.cantidad').value);
9     const precio = parseFloat(row.querySelector('.precio-unitario').
10       value);
11
12     detalles.push({
13       tipoItem, servicioId, medicamentoId,
14       descripcionLinea: row.querySelector('.descripcion').value,
15       cantidad, precioUnitario: precio
16     });
17   });
18
19   const data = await apiPost('/api/medico/facturas', {
20     clienteId: parseInt(document.getElementById('cliente-factura').
21       value),
22     detalles: detalles
23   });
24
25   toast('Factura creada exitosamente', 'success');
26   cargarFacturas();
27 }
```

Listing 5.2: Creación de factura con items dinámicos

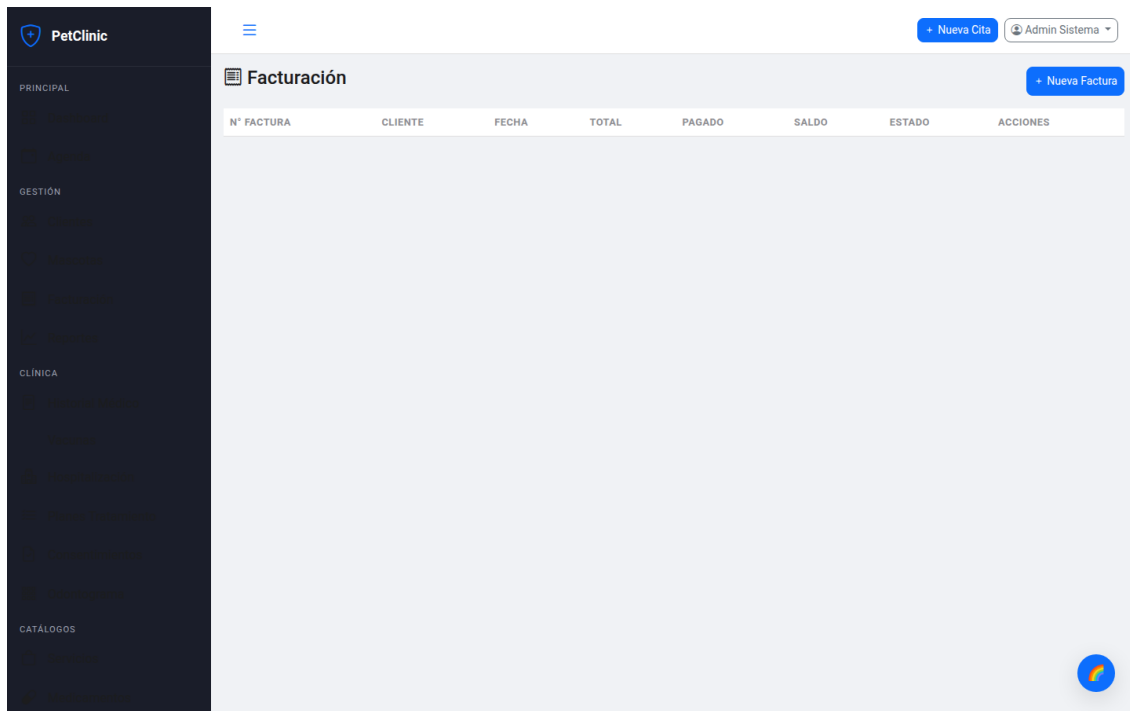


Figura 5.6: Módulo de facturación con items dinámicos

5.7 Reportes

Utiliza Chart.js para mostrar gráficos de ingresos por período con opción de descarga en Excel generado con Apache POI en el backend.

```

1 async function cargarReporte() {
2   const data = await apiGet('/api/medico/reportes/ingresos' +
3     '?inicio=' + document.getElementById('fecha-inicio').value +
4     '&fin=' + document.getElementById('fecha-fin').value);
5
6   new Chart(document.getElementById('chart-ingresos'), {
7     type: 'bar',
8     data: {
9       labels: data.map(d => d.fecha),
10      datasets: [{
11        label: 'Ingresos ($)',
12        data: data.map(d => d.total),
13        backgroundColor: 'rgba(54, 162, 235, 0.5)',
14        borderColor: 'rgba(54, 162, 235, 1)',
15        borderWidth: 1
16      }]
17    },
18    options: {
19      responsive: true,
20      plugins: {

```

```
21     legend: { display: true },
22     tooltip: { callbacks: {
23         label: ctx => '$' + ctx.parsed.y.toFixed(2)
24     }}
25 }
26 }
27 });
28 }
```

Listing 5.3: Renderizado de gráfico de ingresos con Chart.js

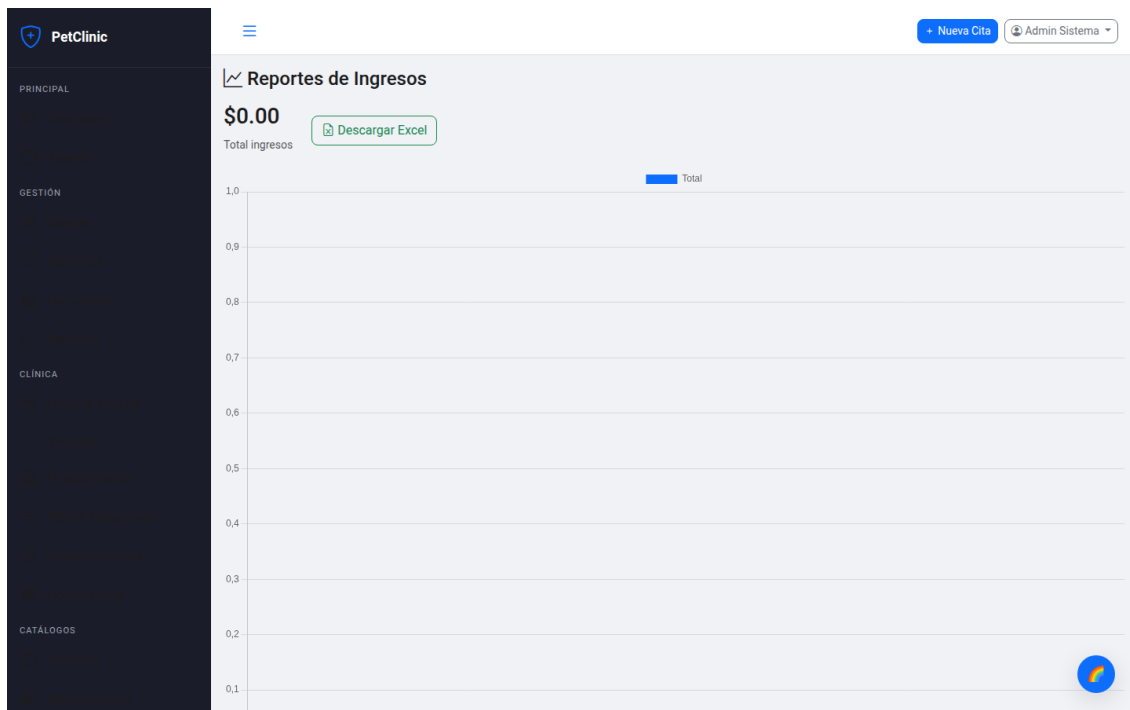


Figura 5.7: Reportes de ingresos con Chart.js

5.8 Odontograma Canino

El odontograma es un diagrama dental interactivo para caninos con 42 dientes numerados según la fórmula dental canina (I 3/3, C 1/1, P 4/4, M 2/3). Permite seleccionar el estado de cada diente (Sano, Tártaro, Caries, Fractura, Ausente, Periodontal, Obturado, Endodoncia) haciendo clic en el diente correspondiente. El estado se persiste en la base de datos y se pueden tener múltiples odontogramas por mascota (uno por consulta).

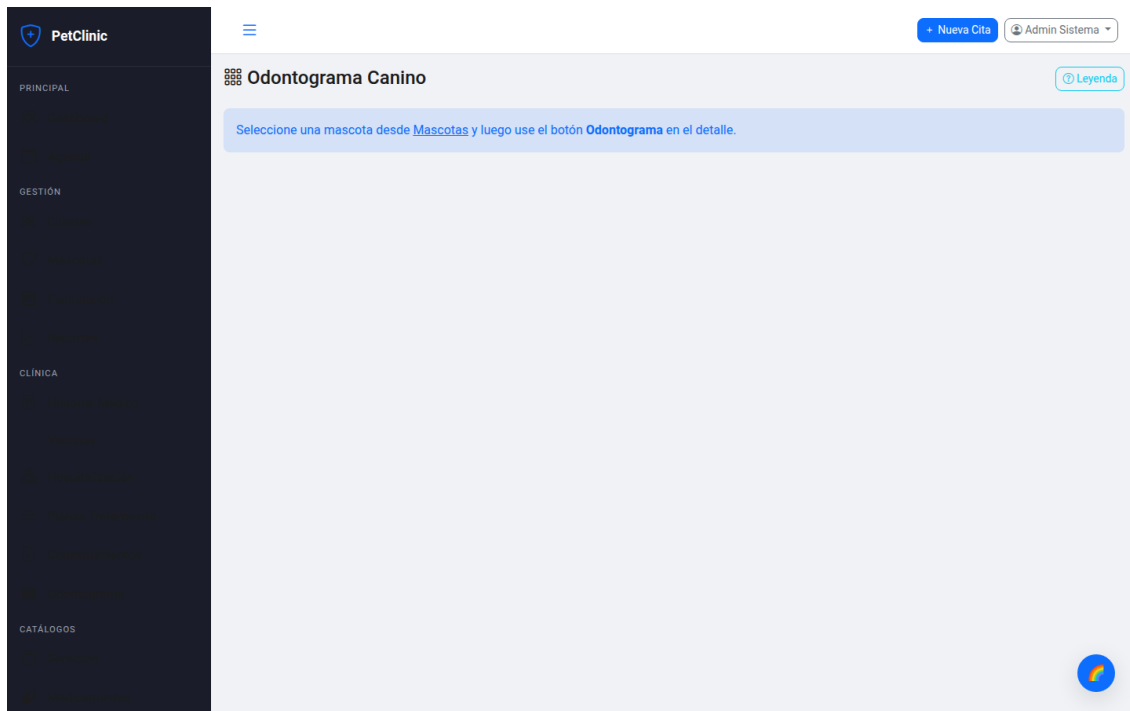


Figura 5.8: Odontograma canino interactivo con 42 dientes y 8 estados

5.9 Portal del Cliente

El portal del cliente permite a los dueños de mascotas consultar sus citas, facturas e historial médico de manera autogestionada, sin necesidad de contacto con el personal de la clínica.

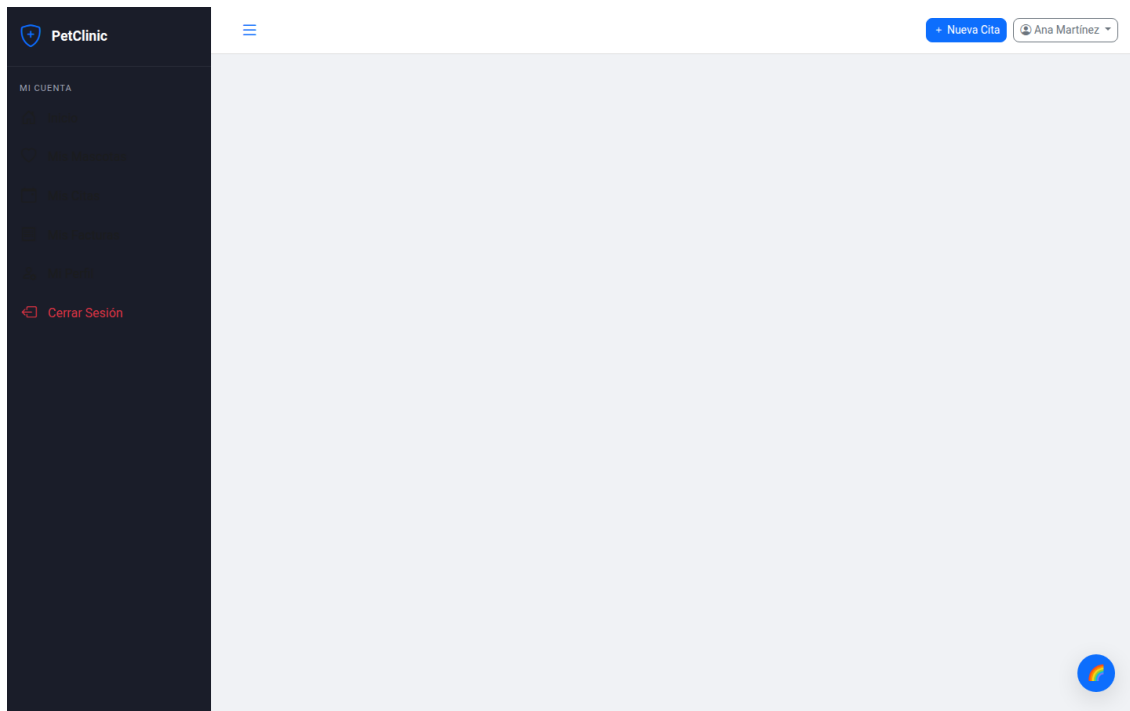


Figura 5.9: Portal del cliente con citas, mascotas y facturas

5.10 Sistema de Temas

El sistema incluye 8 temas de color que se pueden cambiar en tiempo real sin recargar la página mediante el botón flotante en la esquina inferior derecha:

- **Light:** Tema claro predeterminado con fondo blanco y texto oscuro.
- **Dark:** Modo oscuro con fondo #1a1a2e y texto claro.
- **Terracota:** Tonos tierra cálidos con acentos en terracota.
- **Blue Pro:** Azul profesional corporativo.
- **Pink:** Acento rosa para interfaces más suaves.
- **Purple:** Acento púrpura con tonos violeta.
- **Olive:** Tonos verdes oliva.
- **Burn (Sunset):** Ámbar oscuro con degradados de atardecer.

Los temas se implementan mediante variables CSS (*Custom Properties*) y se persisten en `localStorage`. Al cambiar de tema, se actualiza la variable `data-theme` en el elemento `<html>` y FullCalendar se refresca automáticamente.

5.11 API Cliente (api.js)

El módulo `api.js` encapsula toda la comunicación con el backend HTTP mediante `fetch` API con manejo centralizado de errores:

```
1 const API_BASE = '/api';
2
3 function getToken() { return localStorage.getItem('vet_token'); }
4 function getUser() { return JSON.parse(localStorage.getItem('vet_user') || '
  {}'); }
5
6 function authHeaders() {
7   return {
8     'Content-Type': 'application/json',
9     'Authorization': 'Bearer ' + getToken()
10  };
11 }
12
13 async function apiGet(path) {
14   const res = await fetch(API_BASE + path, { headers: authHeaders() });
15   if (!res.ok) {
16     const err = await res.json().catch(() => ({}));
17     throw new Error(err.error || 'Error ' + res.status);
18   }
19   return res.json();
20 }
21
22 async function apiPost(path, body) {
23   const res = await fetch(API_BASE + path, {
24     method: 'POST', headers: authHeaders(),
25     body: JSON.stringify(body)
26   });
27   if (!res.ok) throw new Error((await res.json()).error);
28   return res.json();
29 }
30
31 async function apiPut(path, body) {
32   const res = await fetch(API_BASE + path, {
33     method: 'PUT', headers: authHeaders(),
34     body: JSON.stringify(body)
35   });
36   if (!res.ok) throw new Error((await res.json()).error);
37   return res.json();
38 }
39
40 async function apiDelete(path) {
41   const res = await fetch(API_BASE + path, {
42     method: 'DELETE', headers: authHeaders()
43   });
44   if (!res.ok) throw new Error((await res.json()).error);
45   return res.json();
```

```
46 }
47
48 function fmt(num) { return '$' + Number(num||0).toFixed(2); }
49
50 function toast(msg, type) {
51     const el = document.createElement('div');
52     el.className = 'toast show position-fixed bottom-0 end-0 m-3';
53     el.innerHTML = '<div class="toast-header bg-' + type + ' text-white">'
54         + '<strong>' + type.toUpperCase() + '</strong>'
55         + '<button type="button" class="btn-close btn-close-white" '
56         + 'data-bs-dismiss="toast"></button></div>'
57         + '<div class="toast-body">' + msg + '</div>';
58     document.body.appendChild(el);
59     setTimeout(() => el.remove(), 3000);
60 }
```

Listing 5.4: Cliente HTTP con autenticación JWT

Capítulo 6

Seguridad

6.1 Autenticación JWT

El sistema utiliza JSON Web Tokens (JWT) para la autenticación stateless. La implementación se compone de cuatro clases principales que trabajan en conjunto para validar cada petición HTTP.

6.1.1 JwtTokenProvider

Genera y valida tokens JWT firmados con HMAC-SHA384, incluyendo información del usuario en los claims del token.

```
1 @Component
2 public class JwtTokenProvider {
3     @Value("${app.jwt.secret}")
4     private String jwtSecret;
5
6     @Value("${app.jwt.expiration}")
7     private long jwtExpiration;
8
9     private Key getSigningKey() {
10         return Keys.hmacShaKeyFor(jwtSecret.getBytes());
11     }
12
13     public String generateToken(String email, String rol, Long userId) {
14         Date now = new Date();
15         Date expiry = new Date(now.getTime() + jwtExpiration);
16
17         return Jwts.builder()
18             .subject(email)
19             .claim("rol", rol)
20             .claim("userId", userId)
21             .issuedAt(now)
22             .expiration(expiry)
23             .signWith(getSigningKey())
```

```

24         .compact();
25     }
26
27     public boolean validateToken(String token) {
28         try {
29             Jwts.parser().verifyWith(getSigningKey()).build()
30                 .parseSignedClaims(token);
31             return true;
32         } catch (JwtException | IllegalArgumentException e) {
33             return false;
34         }
35     }
36
37     public String getEmailFromToken(String token) {
38         return Jwts.parser().verifyWith(getSigningKey()).build()
39             .parseSignedClaims(token).getPayload().getSubject();
40     }
41
42     public String getRolFromToken(String token) {
43         return Jwts.parser().verifyWith(getSigningKey()).build()
44             .parseSignedClaims(token).getPayload()
45             .get("rol", String.class);
46     }
47
48     public Long getUserIdFromToken(String token) {
49         return Jwts.parser().verifyWith(getSigningKey()).build()
50             .parseSignedClaims(token).getPayload()
51             .get("userId", Long.class);
52     }
53 }

```

Listing 6.1: JwtTokenProvider: generación y validación de tokens

- **Claims:** subject (email del usuario), rol (ADMIN/VETERINARIO/RECEPCIONISTA/CLIENTE), userId.
- **Firma:** HMAC-SHA384 con clave secreta configurable vía `JWT_SECRET`.
- **Expiración:** 30 minutos (configurable vía `app.jwt.expiration`).

6.1.2 JwtAuthFilter

Filtro que intercepta todas las peticiones HTTP entrantes y valida el token JWT presente en el header `Authorization`. Si el token es válido, establece el contexto de seguridad con los roles del usuario.

```

1 @Component
2 public class JwtAuthFilter extends OncePerRequestFilter {
3
4     @Autowired private JwtTokenProvider jwtProvider;

```

```
5
6  @Override
7  protected void doFilterInternal(HttpServletRequest request,
8                                HttpServletResponse response, FilterChain chain)
9                                throws ServletException, IOException {
10      String header = request.getHeader("Authorization");
11
12      if (header != null && header.startsWith("Bearer ")) {
13          String token = header.substring(7);
14
15          if (jwtProvider.validateToken(token)) {
16              String email = jwtProvider.getEmailFromToken(token);
17              String rol = jwtProvider.getRolFromToken(token);
18              Long userId = jwtProvider.getUserIdFromToken(token);
19
20              UsernamePasswordAuthenticationToken auth =
21                  new UsernamePasswordAuthenticationToken(
22                      email, userId,
23                      List.of(new SimpleGrantedAuthority("ROLE_" + rol))
24                  );
25
26              SecurityContextHolder.getContext().setAuthentication(auth);
27          }
28          chain.doFilter(request, response);
29      }
30  }
```

Listing 6.2: Filtro de autenticación JWT

6.2 Control de Acceso por Roles

6.2.1 SecurityConfig

Configura las reglas de acceso basadas en roles mediante `SecurityFilterChain`:

- **Endpoints públicos:** `/api/auth/**`, `/api/public/**`, `/api/pagos/stripe/webhook`, `/ws/**`, recursos estáticos.
- **ADMIN:** Acceso completo a todos los endpoints, incluyendo administración de usuarios y auditoría.
- **VETERINARIO:** CRUD clínico completo (citás, facturas, historial, vacunas, hospitalización).
- **RECEPCIONISTA:** Sólo lectura en la mayoría de recursos; puede crear clientes y citas.

- **CLIENTE:** Sólo acceso a `/api/portal/**` (perfil, citas, facturas, mascotas).

```

1 @Bean
2 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
3 {
4     http
5         .cors(cors -> cors.configurationSource(corsConfig()))
6         .csrf(csrf -> csrf.disable())
7         .sessionManagement(sm ->
8             sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
9         .authorizeHttpRequests(auth -> auth
10             .requestMatchers("/api/auth/**", "/api/public/**").permitAll()
11             .requestMatchers("/api/pagos/stripe/webhook").permitAll()
12             .requestMatchers("/ws/**").permitAll()
13             .requestMatchers("/api/admin/**").hasRole("ADMIN")
14             .requestMatchers("/api/admin/auditoria")
15                 .hasAnyRole("ADMIN", "VETERINARIO")
16             .requestMatchers(GET, "/api/medico/**")
17                 .hasAnyRole("ADMIN", "VETERINARIO", "RECEPCIONISTA")
18             .requestMatchers("/api/medico/**")
19                 .hasAnyRole("ADMIN", "VETERINARIO")
20             .requestMatchers("/api/portal/**").hasRole("CLIENTE")
21             .anyRequest().authenticated()
22         )
23         .addFilterBefore(jwtAuthFilter,
24             UsernamePasswordAuthenticationFilter.class);
25     return http.build();

```

Listing 6.3: Configuración de seguridad con reglas de acceso

6.3 CustomUserService

Implementa `UserService` para cargar usuarios desde la base de datos. Soporta dos tipos de autenticación:

- **Personal:** Busca en la tabla `veterinarios` (admin, veterinarios, recepcionistas). Usa `VeterinarioRepository.findByEmail()`.
- **Cliente:** Busca en la tabla `clientes` donde `portalActivo = true`. Usa `ClienteRepository.findBy`

```

1 @Override
2 public UserDetails loadUserByUsername(String username)
3     throws UsernameNotFoundException {
4     // Intentar como veterinario (personal)
5     Optional<Veterinario> vet = veterinarioRepo.findByEmail(username);
6     if (vet.isPresent()) {
7         Veterinario v = vet.get();

```

```

8         if (!v.getActivo()) {
9             throw new DisabledException("Usuario desactivado");
10        }
11        return new User(v.getEmail(), v.getPasswordHash(),
12            List.of(new SimpleGrantedAuthority("ROLE_" + v.getRol().name()
13        )));
14    }
15
16    // Intentar como cliente
17    Optional<Cliente> cli = clienteRepo.findByPortalEmail(username);
18    if (cli.isPresent()) {
19        Cliente c = cli.get();
20        if (!c.getPortalActivo()) {
21            throw new DisabledException("Portal desactivado");
22        }
23        return new User(c.getPortalEmail(), c.getPortalPasswordHash(),
24            List.of(new SimpleGrantedAuthority("ROLE_CLIENTE")));
25    }
26
27    throw new UsernameNotFoundException(
28        "Usuario no encontrado: " + username);
29 }

```

Listing 6.4: CustomUserDetailsService

6.4 Auditoría

Cada operación CRUD importante registra un log de auditoría con la siguiente información:

- Acción realizada (CREATE, UPDATE, DELETE).
- Entidad afectada y su ID.
- Descripción detallada de la operación.
- ID y email del veterinario que realizó la acción.
- Dirección IP del cliente (extraída del header X-Forwarded-For o `getRemoteAddr()`).
- Fecha y hora exacta del evento.

```

1 public void log(String accion, String entidad, Long entidadId,
2     String descripcion, Long vetId, String vetEmail) {
3     AuditLog log = new AuditLog();
4     log.setAccion(accion);
5     log.setEntidad(entidad);
6     log.setEntidadId(entidadId);
7     log.setDescripcion(descripcion);

```



```
8 log.setVeterinarioId(vetId);
9 log.setVeterinarioEmail(vetEmail);
10 log.setDireccionIp(obtenerDireccionIp());
11 log.setFechaAccion(LocalDateDateTime.now());
12 auditLogRepo.save(log);
13 }
```

Listing 6.5: Ejemplo de registro de auditoría

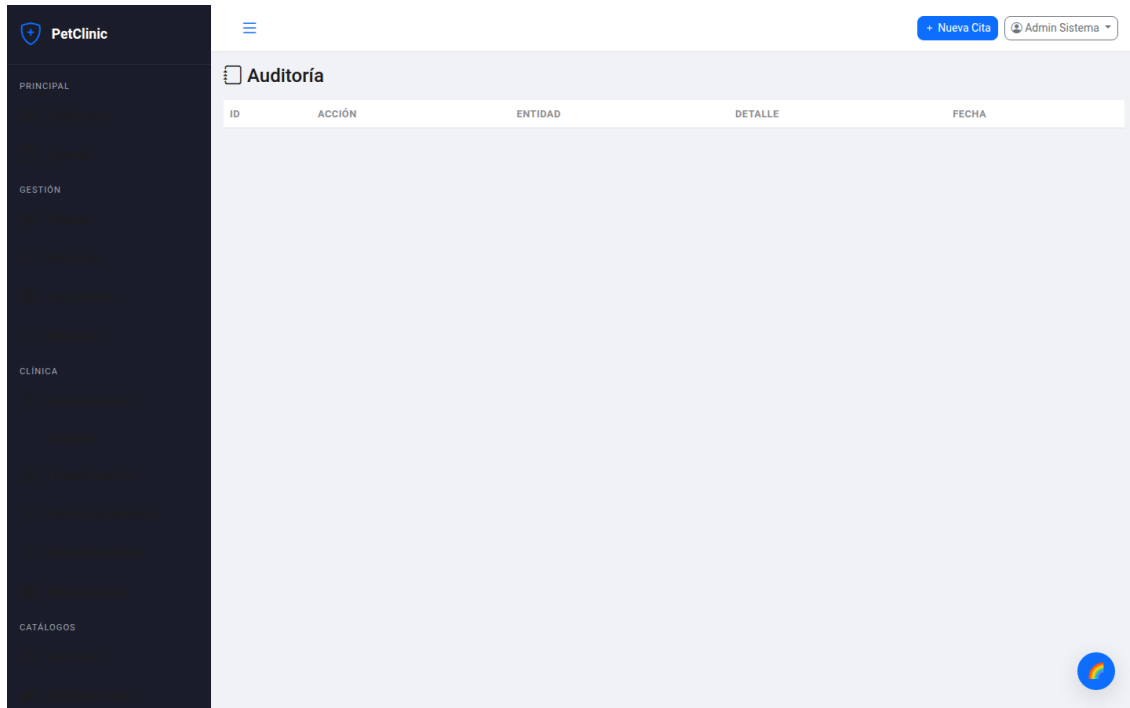


Figura 6.1: Registro de auditoría con detalle de acciones por usuario

6.5 Protección de Contraseñas

Todas las contraseñas se almacenan usando BCrypt mediante `BCryptPasswordEncoder` de Spring Security, con factor de costo 10 (10 rondas de hashing). El `PasswordEncoder` se define como un bean en `SecurityConfig`:

```
1 @Bean
2 public PasswordEncoder passwordEncoder() {
3     return new BCryptPasswordEncoder();
4 }
```

6.6 CORS

Configuración que permite peticiones desde cualquier origen para desarrollo, con soporte para credenciales:

```
1 @Bean
2 public WebMvcConfigurer corsConfigurer() {
3     return new WebMvcConfigurer() {
4         @Override
5         public void addCorsMappings(CorsRegistry registry) {
6             registry.addMapping("/api/**")
7                 .allowedOriginPatterns("*")
8                 .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
9                 .allowedHeaders("*")
10                .allowCredentials(true);
11        }
12    };
13 }
```

Listing 6.6: Configuración CORS

6.7 Protección CSRF

CSRF está deshabilitado porque la aplicación utiliza JWT stateless, lo que hace que los tokens CSRF sean innecesarios. La protección contra CSRF se logra mediante la validación del token JWT en cada petición. En producción, se recomienda agregar `HttpOnly` y `Secure` a las cookies si se usan, pero dado que el token se almacena en `localStorage` del navegador, la protección principal es contra XSS.

Capítulo 7

Módulos del Sistema

7.1 Visión General

PetClinic cuenta con 20 módulos funcionales que cubren todas las áreas operativas de una clínica veterinaria. Cada módulo consta de su controlador REST, servicio, repositorio y vista en el frontend, siguiendo la arquitectura en capas descrita en el capítulo 2.

7.2 Módulo de Veterinarios

Gestión completa del personal de la clínica con roles diferenciados:

- CRUD completo de veterinarios (sólo administradores).
- Roles: ADMIN, VETERINARIO, RECEPCIONISTA.
- Gestión de horarios y duración de turnos.
- Activación/desactivación de cuentas (*soft delete*).
- Cambio de contraseña con verificación de contraseña actual.

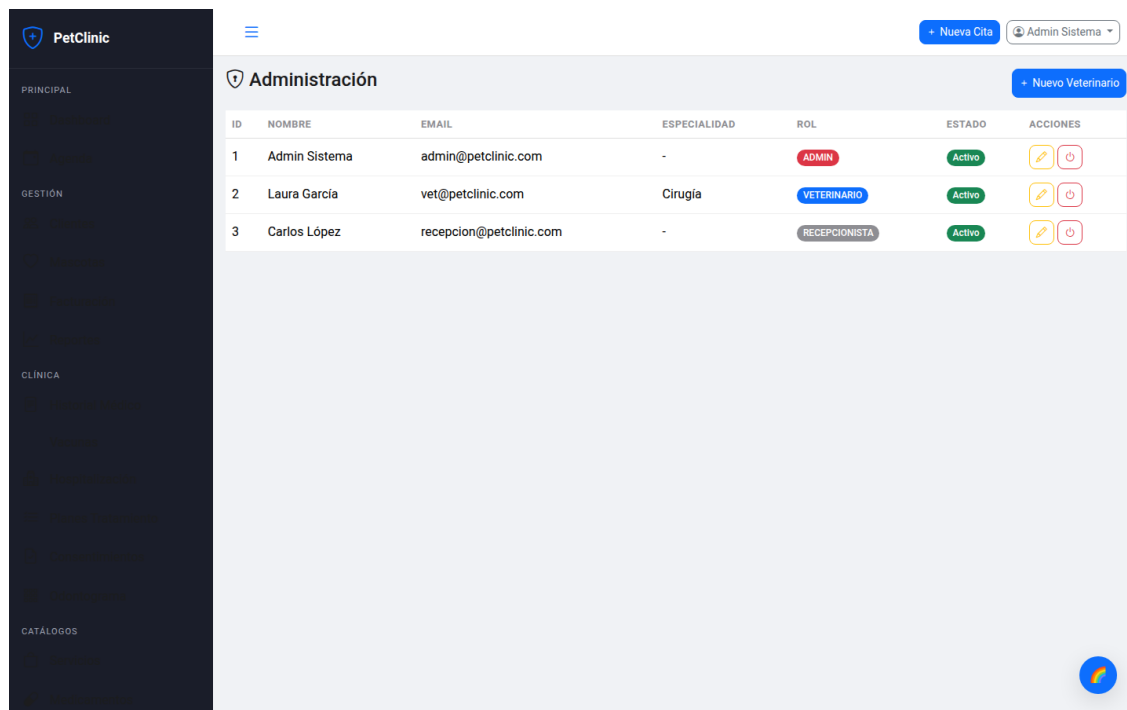


Figura 7.1: Gestión de veterinarios desde el panel de administración

7.3 Módulo de Clientes

Registro y gestión de dueños de mascotas:

- Registro con datos de contacto y dirección.
- Búsqueda por nombre, apellido o email.
- Portal web para clientes (citas, facturas, historial).
- Vista detallada con mascotas asociadas.

7.4 Módulo de Mascotas

Registro detallado de pacientes con información clínica completa:

- Información completa: especie, raza, color, género, peso.
- Control de alergias y condiciones médicas.
- Vista unificada con citas, historial, vacunas y planes.
- Integración con odontograma dental.

7.5 Módulo de Citas

Agendamiento con calendario interactivo y notificaciones en tiempo real:

- Calendario FullCalendar con vistas mensual, semanal y diaria.
- Validación automática de disponibilidad (evita solapamientos usando JPQL).
- Estados: Programada → Confirmada → En Curso → Completada.
- Cancelación con motivo obligatorio.
- Notificaciones en tiempo real vía WebSocket (STOMP sobre SockJS).
- Arrastrar y soltar para reprogramar citas.
- Recordatorio por email al crear la cita.

7.6 Módulo de Facturación

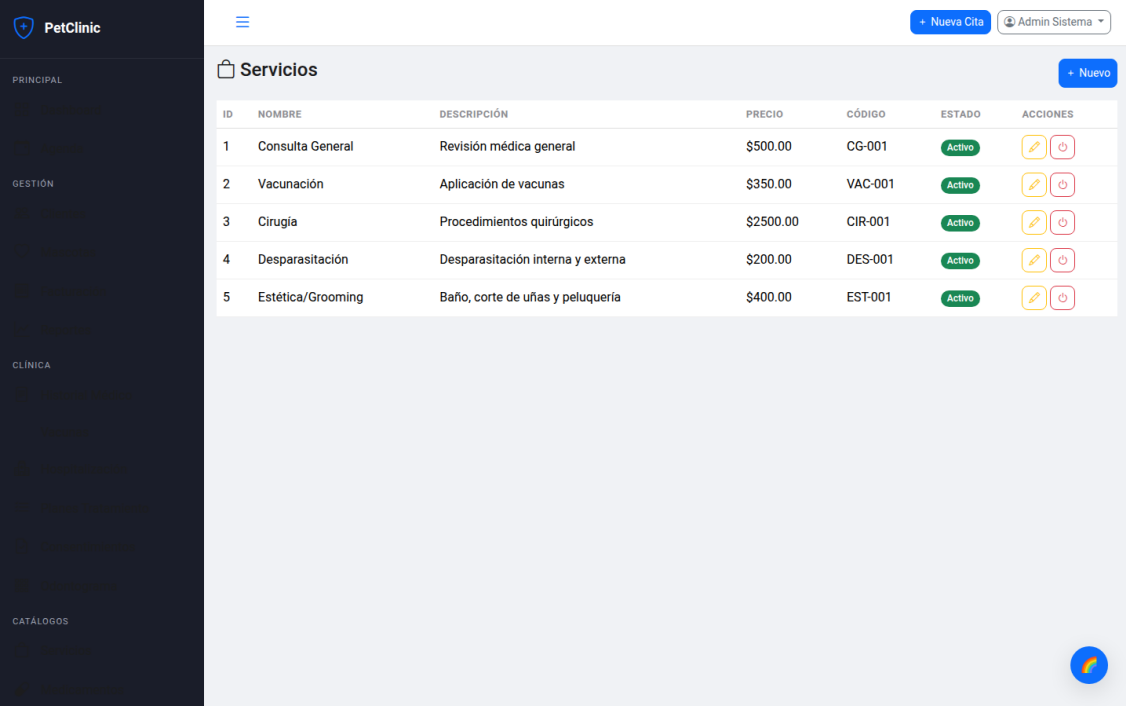
Facturación completa con control de inventario y pagos parciales:

- Facturas con items mixtos (servicios + medicamentos).
- Cálculo automático de subtotales, descuentos y total.
- Registro de pagos parciales con control de saldo pendiente.
- Estados: Pendiente → Pagada Parcial → Pagada → Anulada.
- Descuento automático de stock al incluir medicamentos.
- Generación de PDF con iText 7 (formato profesional con tabla de items).
- Integración con Stripe para pagos en línea.

7.7 Módulo de Servicios

Catálogo de servicios ofrecidos por la clínica:

- CRUD completo con precio base y código interno.
- Activación/desactivación sin eliminar (*soft delete*).



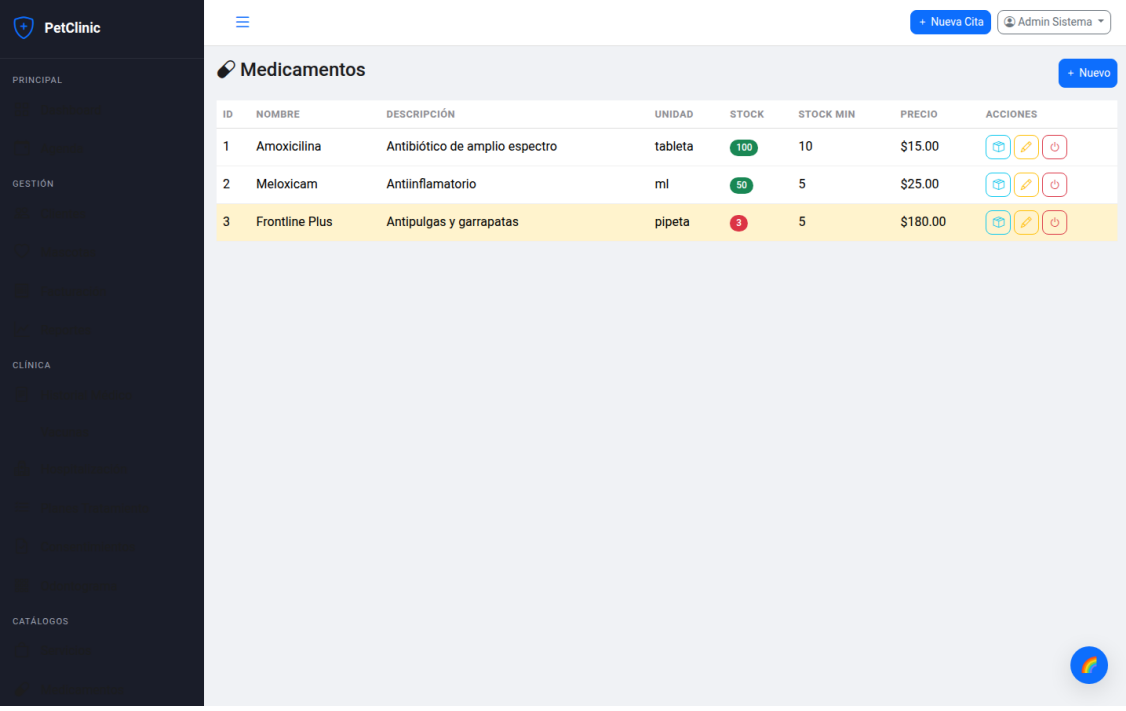
ID	NOMBRE	DESCRIPCIÓN	PRECIO	CÓDIGO	ESTADO	ACCIONES
1	Consulta General	Revisión médica general	\$500.00	CG-001	Activo	 
2	Vacunación	Aplicación de vacunas	\$350.00	VAC-001	Activo	 
3	Cirugía	Procedimientos quirúrgicos	\$2500.00	CIR-001	Activo	 
4	Desparasitación	Desparasitación interna y externa	\$200.00	DES-001	Activo	 
5	Estética/Grooming	Baño, corte de uñas y peluquería	\$400.00	EST-001	Activo	 

Figura 7.2: Catálogo de servicios con precios y activación

7.8 Módulo de Medicamentos e Inventario

Control de stock con trazabilidad completa de movimientos:

- CRUD de medicamentos con unidad de medida y presentación.
- Control de stock mínimo con alertas visuales (stock bajo resaltado en rojo).
- Trazabilidad: cada movimiento registra tipo (entrada, salida, ajuste), cantidad, fecha y usuario.
- Descuento automático al facturar medicamentos.
- Vista de stock bajo (medicamentos con stock menor o igual al mínimo configurado).



ID	NOMBRE	DESCRIPCIÓN	UNIDAD	STOCK	STOCK MIN	PRECIO	ACCIONES
1	Amoxicilina	Antibiótico de amplio espectro	tableta	100	10	\$15.00	  
2	Meloxicam	Antiinflamatorio	ml	90	5	\$25.00	  
3	Frontline Plus	Antipulgas y garrapatas	pipeta	3	5	\$180.00	  

Figura 7.3: Inventario de medicamentos con control de stock

7.9 Módulo de Historial Médico

Notas detalladas de consulta por mascota con orden cronológico:

- Registro de motivo, diagnóstico, procedimiento y medicación recetada.
- Vinculación a citas específicas para contexto completo.
- Historial cronológico completo por mascota con búsqueda por fecha.

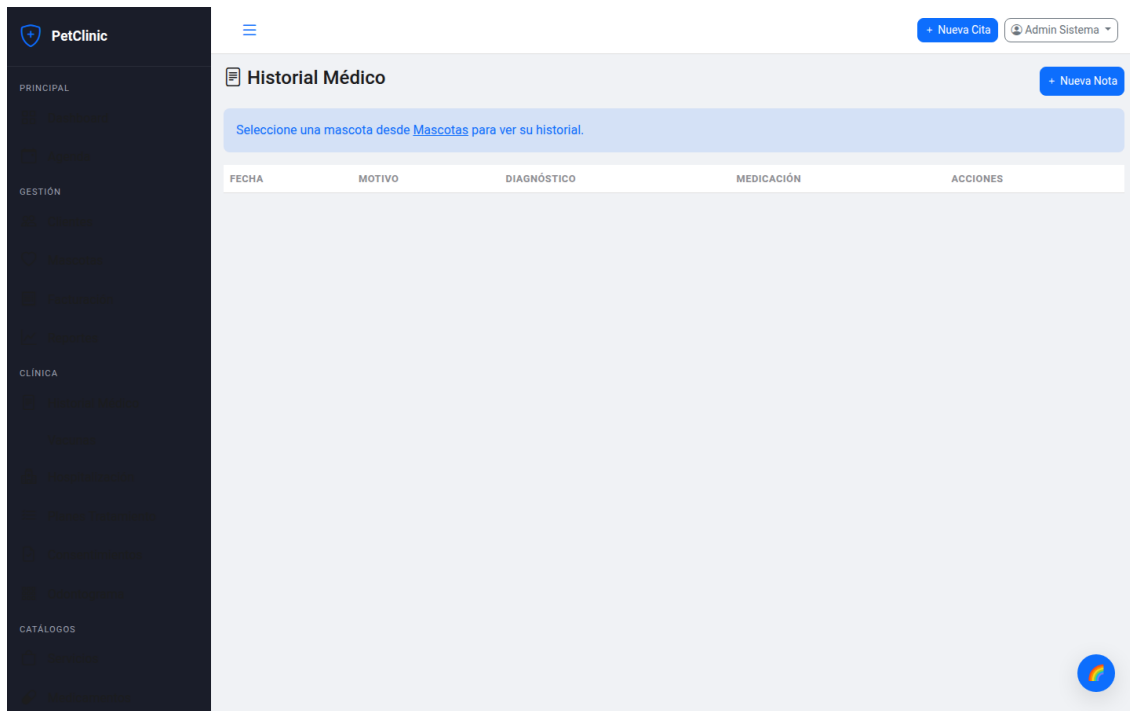


Figura 7.4: Historial médico de mascota con notas de consulta

7.10 Módulo de Vacunas

Control de vacunación con alertas automáticas por email:

- Registro con nombre de vacuna, fecha de aplicación y próxima dosis.
- Control de lote y fabricante para trazabilidad.
- Alertas automáticas por email: vacunas próximas a vencer (08:00) y vencidas (09:00).
- Tareas programadas con @Scheduled y cron expressions.
- Integración con JavaMailSender para envío de correos HTML con plantillas.

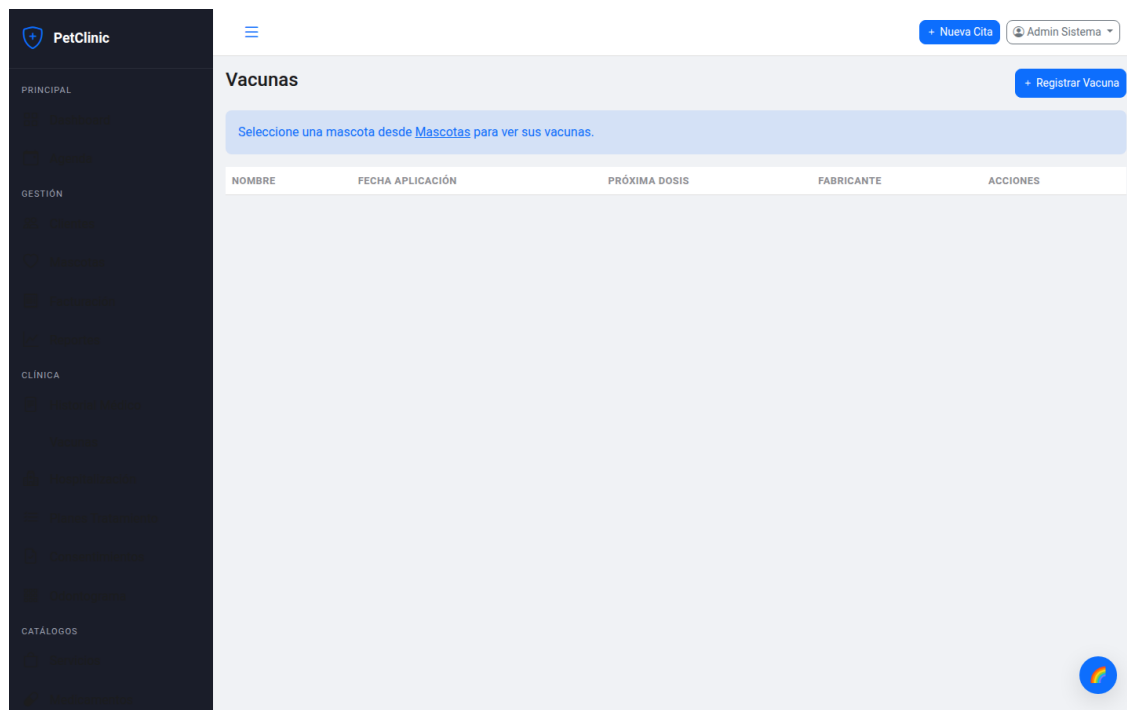


Figura 7.5: Registro de vacunas por mascota con alertas

7.11 Módulo de Hospitalización

Gestión de pacientes hospitalizados con control de ingresos y altas:

- Ingreso con asignación de jaula y motivo de hospitalización.
- Control de check-in / check-out con fecha y hora exacta.
- Estado: Hospitalizado → Dado de Alta.
- Notas de evolución durante la estancia.

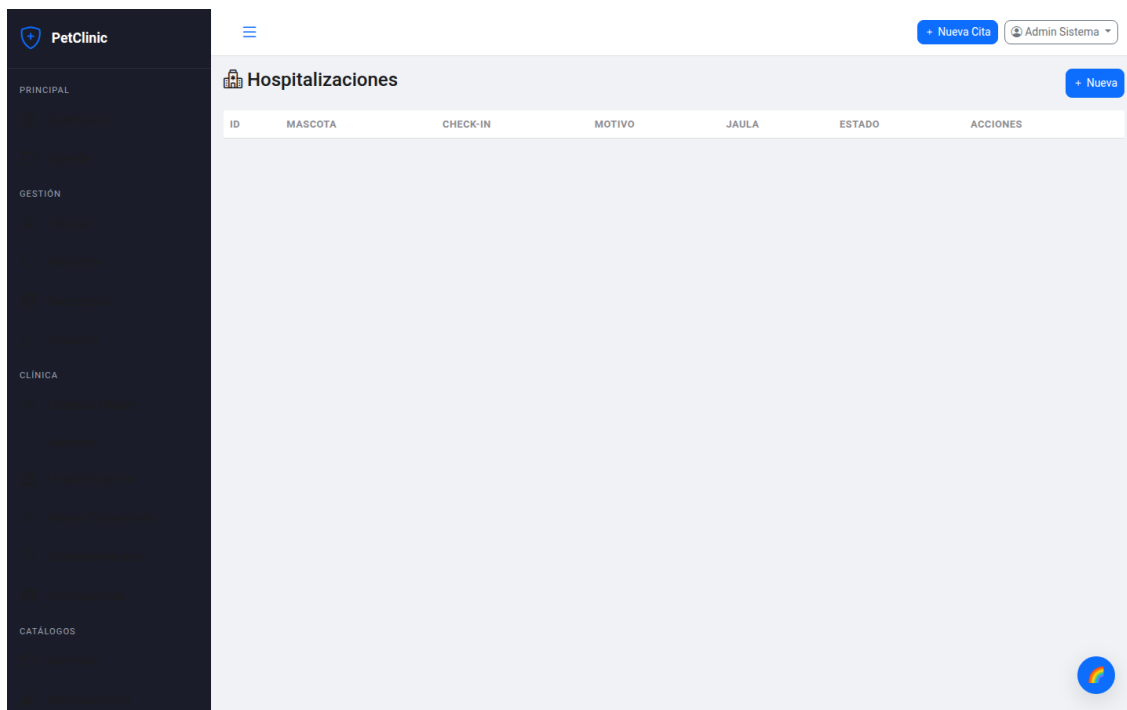


Figura 7.6: Pacientes hospitalizados con control de ingresos

7.12 Módulo de Planes de Tratamiento

Planes con pasos secuenciados para seguimiento de tratamientos:

- Planes con título, descripción y costo estimado total.
- Pasos secuenciados con orden, servicio asociado y costo individual.
- Seguimiento de avance por paso: Pendiente → En Proceso → Completado → Omitido.
- Estados del plan: Activo, Completado, Cancelado.

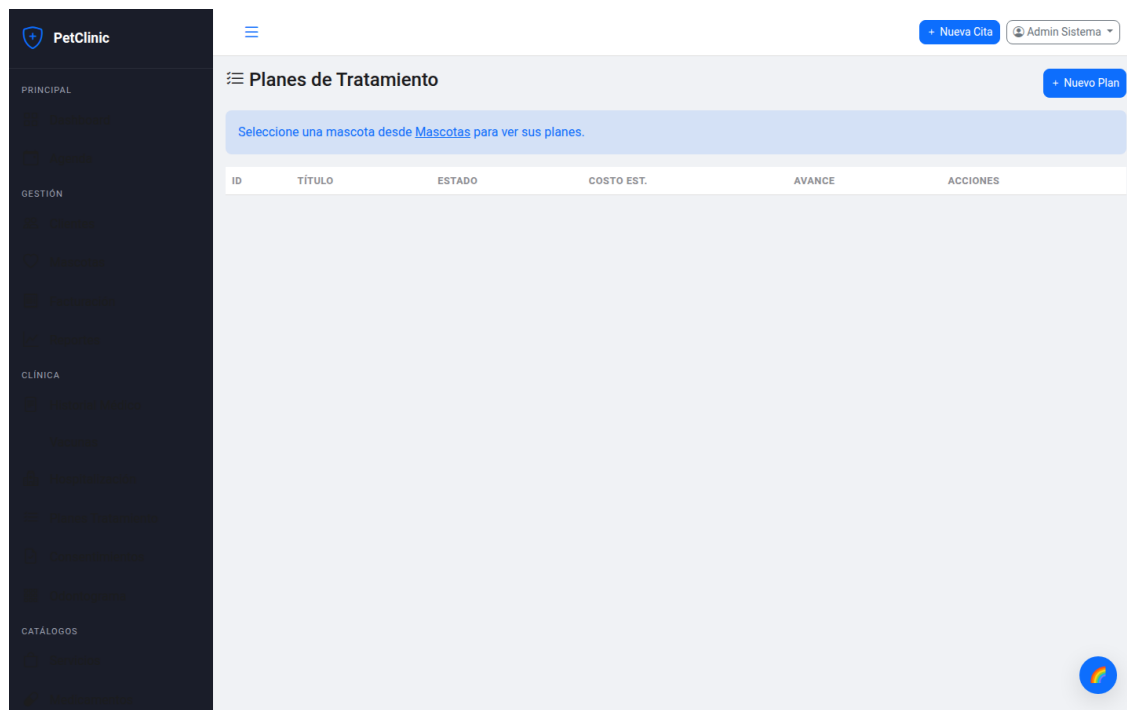


Figura 7.7: Plan de tratamiento con pasos secuenciados y avance

7.13 Módulo de Consentimientos Informados

Documentos legales para procedimientos veterinarios:

- Creación de documentos con título, tipo de procedimiento y contenido detallado.
- Firma digital con registro de fecha, nombre completo del firmante y relación con la mascota.
- Generación de PDF profesional para impresión y respaldo.
- Control de estado: Pendiente de Firma / Firmado.

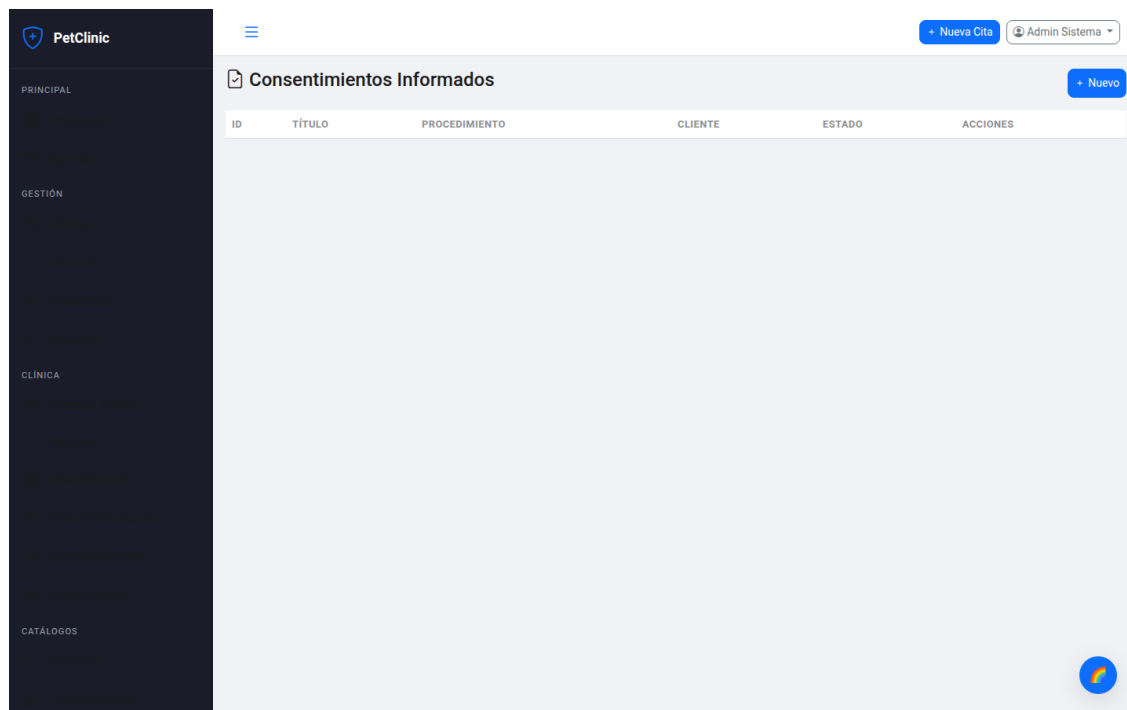


Figura 7.8: Consentimiento informado con firma digital y PDF

7.14 Módulo de Odontograma

Diagrama dental canino interactivo implementado con HTML Canvas:

- 42 dientes numerados según fórmula dental canina (I 3/3, C 1/1, P 4/4, M 2/3).
- Estados: SANO, TÁRTARO, CARIES, FRACTURA, AUSENTE, PERIODONTAL, OBTURADO, ENDODONCIA.
- Cada estado se representa con un color diferente para identificación visual rápida.
- Interfaz interactiva: seleccionar estado en la leyenda y hacer clic en el diente.
- Persistencia de múltiples odontogramas por mascota (uno por consulta dental).

7.15 Módulo de Pagos Online (Stripe)

Integración con Stripe Payment Intents para pagos en línea:

- Creación de intención de pago vinculada a una factura específica.
- Modal simulado cuando Stripe no está configurado (*sandbox mode*).

- Webhook `/api/pagos/stripe/webhook` para confirmación asíncrona de pagos.
- Actualización automática del estado de la factura al confirmar el pago.

7.16 Módulo de Reserva Pública

Permite a clientes potenciales agendar citas sin autenticación:

- Listado de veterinarios disponibles con horarios y especialidades.
- Visualización de slots disponibles por fecha y veterinario.
- Validación de email del cliente contra la base de datos existente.
- Creación de cita como visitante sin necesidad de registro.

7.17 Módulo de Reportes

Análisis de ingresos con visualización gráfica y exportación:

- Dashboard con 7 indicadores clave del sistema.
- Reporte de ingresos por período con gráfico de barras (Chart.js).
- Exportación a Excel con Apache POI para análisis externo.
- Filtro por rango de fechas personalizado.

7.18 Módulo de Auditoría

Registro de todas las acciones importantes del sistema para cumplimiento y trazabilidad:

- Log de acciones CREATE, UPDATE, DELETE con tipo de operación.
- Detalle de entidad afectada y usuario responsable.
- Registro de dirección IP del cliente.
- Visualización en tabla cronológica con filtros por entidad y fecha.

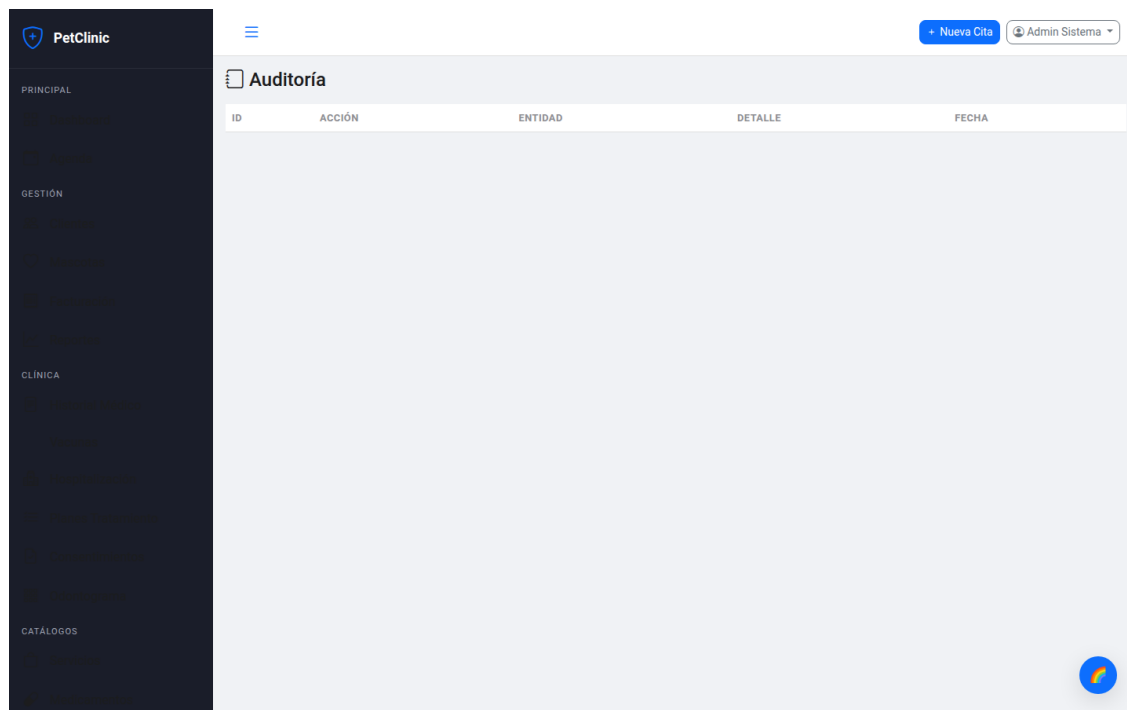


Figura 7.9: Log de auditoría con acciones del sistema

7.19 Módulo de Configuración

Panel de configuración general del sistema:

- Configuración de datos de la clínica (nombre, dirección, teléfono, email).
- Configuración de parámetros de facturación (impuestos, formato de número).
- Configuración de notificaciones (activar/desactivar alertas por email).

Figura 7.10: Panel de configuración general del sistema

7.20 Módulo de Notificaciones

Sistema de alertas multicanal con tareas programadas:

- Alertas de vacunación por email (vacunas próximas a vencer y vencidas).
- Notificaciones automáticas diarias vía `@Scheduled` a las 08:00 y 09:00.
- Recordatorios de citas por email al momento de creación.
- Notificaciones de facturación (factura creada, pago recibido).
- Integración con `JavaMailSender` (SMTP) y plantillas HTML personalizables.

```

1 @Component
2 public class VacunaScheduler {
3
4     @Scheduled(cron = "0 0 8 * * ?") // 08:00 todos los dias
5     public void alertarProximasVacunas() {
6         LocalDate hoy = LocalDate.now();
7         LocalDate enDosSemanas = hoy.plusDays(14);
8         List<Vacuna> proximas = vacunaRepo
9             .findByFechaProximaDosisBetween(hoy, enDosSemanas);
10        for (Vacuna v : proximas) {

```

```

11         notificacionService.enviarAlertaVacunacion(v);
12     }
13 }
14
15 @Scheduled(cron = "0 0 9 * * ?") // 09:00 todos los dias
16 public void alertarVacunasVencidas() {
17     List<Vacuna> vencidas = vacunaRepo
18         .findByFechaProximaDosisBefore(LocalDate.now());
19     for (Vacuna v : vencidas) {
20         notificacionService.enviarAlertaVacunacion(v);
21     }
22 }
23 }

```

Listing 7.1: Programación de alertas de vacunación

7.21 Sistema de Temas

Personalización visual con 8 temas de color intercambiables en tiempo real:

- Implementado mediante variables CSS (*Custom Properties*) en el elemento `:root`.
- Persistencia en `localStorage` para recordar el tema entre sesiones.
- Aplicación en tiempo real sin recarga de página.
- Botón flotante en esquina inferior derecha con selector de temas.
- Integración con FullCalendar (actualiza colores del calendario al cambiar tema).

7.22 Credenciales de Prueba

Rol	Email	Password	Descripción
ADMIN	admin@petclinic.com	Admin123	Acceso total al sistema
VETERINARIO	vet@petclinic.com	Vet123	Gestión clínica completa
RECEPCIONISTA	Arecepcion@petclinic.com	Recep123	Lectura y creación básica
CLIENTE	ana@email.com	Ana123	Portal del cliente

Capítulo 8

Pruebas

8.1 Enfoque de Pruebas

El proyecto utiliza pruebas de integración con JUnit 5 y Spring Boot Test. Las pruebas se ejecutan contra una base de datos H2 en memoria, lo que permite validar el funcionamiento completo del sistema sin dependencias externas. Se siguen las siguientes prácticas:

- **Orden de ejecución:** Las pruebas se ejecutan en orden (`@TestMethodOrder(MethodName)` o `@Order`) para mantener consistencia en el estado de la base de datos.
- **Autenticación previa:** Cada clase de prueba autentica primero al usuario correspondiente (admin, vet, cliente) y reutiliza el token JWT obtenido.
- **Data inicial:** Los datos de prueba se cargan mediante `DataInitializer` al iniciar el contexto de Spring.
- **Configuración aislada:** Archivo `application.properties` específico para pruebas con H2 y logging reducido.

8.2 Ejecución de Pruebas

```
1 mvn test
```

Listing 8.1: Comando para ejecutar todas las pruebas

Para ejecutar una prueba específica:

```
1 mvn test -Dtest=VeterinarySystemIntegrationTest
2 mvn test -Dtest=NewFeaturesIntegrationTest#testCreateFactura
```

Listing 8.2: Ejecutar prueba individual

8.3 Clases de Prueba

8.3.1 VeterinarySystemIntegrationTest

Contiene 15 pruebas de integración que cubren el flujo básico del sistema:

1. `testLoginAdmin`: Inicio de sesión como administrador, obtención de JWT y verificación de claims.
2. `testLoginVet`: Inicio de sesión como veterinario con rol `VETERINARIO`.
3. `testDashboard`: Acceso al dashboard con token de administrador.
4. `testGetClientes`: Listado paginado de clientes.
5. `testGetMascotas`: Listado de mascotas filtradas por cliente.
6. `testCreateCita`: Creación de cita con validación de disponibilidad.
7. `testGetServicios`: Consulta de servicios activos.
8. `testGetMedicamentos`: Consulta de medicamentos activos.
9. `testCreateFactura`: Creación de factura con descuento automático de stock.
10. `testGetFacturaPdf`: Generación y verificación de PDF de factura.
11. `testHistorialMedico`: Consulta de historial médico por mascota.
12. `testHospitalizaciones`: Listado de hospitalizaciones activas.
13. `testPortalAccessDenied`: Verificación de que un veterinario no accede al portal de cliente.
14. `testLoginCliente`: Inicio de sesión como cliente y acceso al portal.
15. `testDeleteCita`: Eliminación lógica de cita.

```
1 @SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
2 @TestMethodOrder(MethodName.class)
3 class VeterinarySystemIntegrationTest {
4
5     @Autowired private TestRestTemplate restTemplate;
6     private static String adminToken;
7     private static Long clienteId;
8     private static Long facturaId;
9
10    @Test
11    @Order(1)
```

```
12 void testLoginAdmin() {
13     LoginRequest req = new LoginRequest("admin@petclinic.com",
14         "Admin123", "PERSONAL");
15     ResponseEntity<LoginResponse> res = restTemplate.postForEntity(
16         "/api/auth/login", req, LoginResponse.class);
17     assertEquals(HttpStatus.OK, res.getStatusCode());
18     assertNotNull(res.getBody().getToken());
19     adminToken = res.getBody().getToken();
20 }
21
22 @Test
23 @Order(2)
24 void testDashboard() {
25     HttpHeaders headers = new HttpHeaders();
26     headers.setBearerAuth(adminToken);
27     HttpEntity<Void> entity = new HttpEntity<>(headers);
28
29     ResponseEntity<Map> res = restTemplate.exchange(
30         "/api/medico/dashboard", HttpMethod.GET, entity, Map.class);
31     assertEquals(HttpStatus.OK, res.getStatusCode());
32     assertNotNull(res.getBody().get("totalClientes"));
33 }
34 }
```

Listing 8.3: Ejemplo: autenticación como admin y prueba de dashboard

8.3.2 NewFeaturesIntegrationTest

Contiene 15 pruebas adicionales que cubren funcionalidades nuevas y endpoints públicos:

1. `testLoginAdmin` / `testLoginVet`: Pruebas de autenticación para ambos roles.
2. `testPublicListVeterinarios`: Listado público de veterinarios activos sin autenticación.
3. `testPublicSlots`: Visualización de slots disponibles para una fecha y veterinario.
4. `testPublicSlotsInvalidVet`: Veterinario inválido retorna 404.
5. `testPublicCreateCita`: Creación de cita pública como visitante.
6. `testPublicCreateCitaWrongEmail`: Email incorrecto retorna 400.
7. `testCreateFacturaForPayment`: Creación de factura para prueba de pago.
8. `testCreatePaymentIntentUnauthenticated`: Pago requiere autenticación (retorna 401).
9. `testCreatePaymentIntent`: Creación de intención de pago Stripe.
10. `testCreatePaymentIntentInvalidFactura`: Factura inválida retorna 400.

11. `testVacunasForMascota`: Listado de vacunas por mascota.
12. `testCitaCreateTriggersNotification`: Creación de cita dispara notificación programada.
13. `testAllNewServiceBeansExist`: Verificación de que todos los beans de servicios se cargan correctamente.
14. `testPublicEndpointsNoAuthRequired`: Endpoints públicos accesibles sin token.
15. `testPaymentEndpointRequiresAuth`: Endpoint de pago protegido correctamente.

8.4 Configuración de Pruebas

Las pruebas utilizan una configuración específica en `src/test/resources/application.properties`:

```
1 # Base de datos en memoria H2
2 spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
3 spring.datasource.driver-class-name=org.h2.Driver
4 spring.jpa.hibernate.ddl-auto=create-drop
5
6 # JWT para pruebas (clave fija)
7 app.jwt.secret=TestSecretKeyForTestingPurposesOnly2024!!
8 app.jwt.expiration=3600000
9
10 # Email desactivado en pruebas
11 spring.mail.port=-1
12
13 # Logging reducido para pruebas
14 logging.level.com.veterinary=WARN
```

Listing 8.4: Configuración para pruebas de integración

8.5 Ejemplo: Prueba de Creación de Factura con Descuento de Stock

```
1 @Test
2 @Order(9)
3 void testCreateFactura() {
4     HttpHeaders headers = new HttpHeaders();
5     headers.setBearerAuth(adminToken);
6     headers.setContentType(MediaType.APPLICATION_JSON);
7
8     String body = ""
9     {
10         "clienteId": %d,
11         "detalles": [
```

```
12         {"tipoItem": "SERVICIO", "servicioId": 1,
13          "descripcionLinea": "Consulta General",
14          "cantidad": 1, "precioUnitario": 500},
15         {"tipoItem": "MEDICAMENTO", "medicamentoId": 1,
16          "descripcionLinea": "Amoxicilina",
17          "cantidad": 2, "precioUnitario": 15}
18     ]
19 }
20 """.formatted(clienteId);
21
22 HttpEntity<String> entity = new HttpEntity<>(body, headers);
23 ResponseEntity<Map> res = restTemplate.exchange(
24     "/api/medico/facturas", HttpMethod.POST, entity, Map.class);
25
26 assertEquals(HttpStatus.OK, res.getStatusCode());
27 assertNotNull(res.getBody().get("numeroFactura"));
28 assertEquals(530.0, (Double) res.getBody().get("total"), 0.01);
29 facturaId = ((Number) res.getBody().get("id")).longValue();
30 }
```

Listing 8.5: Prueba de integración para creación de factura

8.6 Ejemplo: Prueba de Endpoints Públicos

```
1 @Test
2 void testPublicSlots() {
3     ResponseEntity<String> res = restTemplate.getForEntity(
4         "/api/public/slots?vetId=1&fecha="
5         + LocalDate.now().plusDays(1).toString(),
6         String.class);
7     assertEquals(HttpStatus.OK, res.getStatusCode());
8     assertTrue(res.getBody().contains("hora"));
9     assertTrue(res.getBody().contains("disponible"));
10 }
11
12 @Test
13 void testPublicEndpointsNoAuthRequired() {
14     ResponseEntity<String> res = restTemplate.getForEntity(
15         "/api/public/veterinarios", String.class);
16     assertEquals(HttpStatus.OK, res.getStatusCode());
17 }
```

Listing 8.6: Prueba de agenda pública sin autenticación

8.7 Cobertura de Pruebas

Las 30 pruebas de integración cubren las siguientes áreas del sistema:

- **Autenticación (6):** Login como ADMIN, VETERINARIO, RECEPCIONISTA y CLIENTE; validación de credenciales incorrectas; protección de endpoints.
- **CRUD (8):** Creación, lectura, actualización y eliminación de entidades principales (clientes, mascotas, citas, facturas).
- **Reglas de negocio (5):** Validación de disponibilidad de citas, descuento de stock en facturación, control de acceso por roles, validación de email en reserva pública.
- **Integración (4):** WebSocket, notificaciones por email, generación de PDF con iText, exportación Excel.
- **Pagos (3):** Creación de Payment Intent, webhook de Stripe, protección de endpoint de pago.
- **Reserva pública (3):** Listado de veterinarios, slots disponibles, creación de cita sin autenticación.
- **Existencia de beans (1):** Verificación de que todos los servicios se cargan en el contexto de Spring.

Capítulo 9

Despliegue

9.1 Requisitos del Servidor

- Java 21 o superior (OpenJDK 21 LTS recomendado).
- PostgreSQL 14 o superior.
- Maven 3.8+ (para compilación desde código fuente).
- Al menos 512 MB de RAM para la aplicación (1 GB recomendado).
- Conexión SMTP saliente para notificaciones por email (opcional).
- Cuenta de Stripe con API keys para pagos en línea (opcional).

9.2 Configuración de Base de Datos

9.2.1 Crear Base de Datos PostgreSQL

```
1 sudo -u postgres psql -c "CREATE DATABASE veterinary_db;"
2 sudo -u postgres psql -c "CREATE USER veterinary_user WITH PASSWORD '
  veterinary_pass';"
3 sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE veterinary_db
  TO veterinary_user;"
```

Listing 9.1: Creación de base de datos y usuario en PostgreSQL

9.2.2 Verificar Conexión

```
1 psql -U veterinary_user -d veterinary_db -h localhost
```

Listing 9.2: Verificar que la base de datos es accesible

9.3 Ejecución en Desarrollo

```
1 mvn spring-boot:run
```

Listing 9.3: Ejecutar la aplicación en modo desarrollo

La aplicación estará disponible en `http://localhost:8090`. La base de datos se crea y puebla automáticamente al primer inicio gracias a `spring.jpa.hibernate.ddl-auto=update` y el `DataInitializer`.

9.4 Compilación para Producción

```
1 mvn clean package -DskipTests
```

Listing 9.4: Compilar y empaquetar como JAR

Genera el archivo `target/veterinary-clinic-management-system-1.0.0.jar`.

```
1 export JWT_SECRET="clave-secreta-muy-segura-12345"
2 export JWT_EXPIRATION=1800000
3 export MAIL_HOST=smtp.gmail.com
4 export MAIL_USERNAME=notificaciones@petclinic.com
5 export MAIL_PASSWORD=contrasena-de-app
6 export STRIPE_API_KEY=sk_live_...
7 export STRIPE_WEBHOOK_SECRET=whsec_...
8
9 java -jar target/veterinary-clinic-management-system-1.0.0.jar
```

Listing 9.5: Ejecutar JAR compilado con variables de entorno

9.5 Despliegue con Docker

9.5.1 Dockerfile

```
1 FROM maven:3.9-eclipse-temurin-21 AS builder
2 WORKDIR /app
3 COPY pom.xml .
4 RUN mvn dependency:go-offline
5 COPY src ./src
6 RUN mvn clean package -DskipTests
7
8 FROM openjdk:21-jdk-slim
9 WORKDIR /app
10 COPY --from=builder /app/target/*.jar app.jar
11 EXPOSE 8090
12 ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Listing 9.6: Dockerfile multi-etapa para imagen optimizada

9.5.2 Docker Compose

```
1 version: '3.8'
2 services:
3   postgres:
4     image: postgres:14
5     environment:
6       POSTGRES_DB: veterinary_db
7       POSTGRES_USER: veterinary_user
8       POSTGRES_PASSWORD: veterinary_pass
9     ports:
10      - "5432:5432"
11     volumes:
12      - postgres_data:/var/lib/postgresql/data
13
14   app:
15     build: .
16     ports:
17      - "8090:8090"
18     environment:
19       SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/veterinary_db
20       SPRING_DATASOURCE_USERNAME: veterinary_user
21       SPRING_DATASOURCE_PASSWORD: veterinary_pass
22       JWT_SECRET: clave-secreta-docker-2024
23       JWT_EXPIRATION: 1800000
24     depends_on:
25      - postgres
26
27 volumes:
28   postgres_data:
```

Listing 9.7: Docker Compose con PostgreSQL y aplicación

```
1 docker-compose build
2 docker-compose up -d
3 docker-compose logs -f app # Ver logs en tiempo real
```

Listing 9.8: Construir y ejecutar con Docker Compose

9.6 Despliegue con systemd (Producción Linux)

Para ejecutar la aplicación como servicio del sistema:

```
1 [Unit]
2 Description=PetClinic Veterinary Clinic
3 After=network.target postgresql.service
4
5 [Service]
6 Type=simple
7 User=petclinic
```

```
8 WorkingDirectory=/opt/petclinic
9 Environment="JWT_SECRET=..."
10 Environment="JWT_EXPIRATION=1800000"
11 Environment="MAIL_HOST=smtp.gmail.com"
12 ExecStart=/usr/bin/java -jar /opt/petclinic/app.jar
13 Restart=on-failure
14 RestartSec=10
15
16 [Install]
17 WantedBy=multi-user.target
```

Listing 9.9: Archivo de servicio systemd

```
1 sudo cp petclinic.service /etc/systemd/system/
2 sudo systemctl daemon-reload
3 sudo systemctl enable petclinic
4 sudo systemctl start petclinic
5 sudo systemctl status petclinic
```

Listing 9.10: Instalar y gestionar el servicio

9.7 Proxy Inverso con Nginx

Para entornos de producción con dominio propio:

```
1 server {
2     listen 80;
3     server_name petclinic.midominio.com;
4
5     location / {
6         proxy_pass http://localhost:8090;
7         proxy_set_header Host $host;
8         proxy_set_header X-Real-IP $remote_addr;
9         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
10        proxy_set_header X-Forwarded-Proto $scheme;
11    }
12
13    location /ws {
14        proxy_pass http://localhost:8090;
15        proxy_http_version 1.1;
16        proxy_set_header Upgrade $http_upgrade;
17        proxy_set_header Connection "upgrade";
18        proxy_set_header Host $host;
19    }
20 }
```

Listing 9.11: Configuración de Nginx como proxy inverso

9.8 Variables de Entorno

El sistema soporta configuración mediante variables de entorno para entornos productivos, lo que permite despliegues sin modificar archivos de configuración:

Variable	Descripción	Valor por Defecto
JWT_SECRET	Clave secreta para firmar tokens JWT (HMAC-SHA384)	(valor interno)
JWT_EXPIRATION	Duración del token en milisegundos	1800000 (30 min)
MAIL_HOST	Servidor SMTP para envío de correos	smtp.gmail.com
MAIL_PORT	Puerto SMTP	587
MAIL_USERNAME	Usuario SMTP	(vacío)
MAIL_PASSWORD	Contraseña SMTP	(vacío)
STRIPE_API_KEY	Clave API de Stripe (pk_/sk_)	sk_test_placeholder
STRIPE_WEBHOOK_SECRET	Clave de webhook Stripe	whsec_placeholder
FRONTEND_URL	URL del frontend para enlaces en emails	http://localhost:8090
UPLOAD_DIR	Directorio de subida de archivos	./uploads

9.9 Estructura Completa del Proyecto

```
veterinary-clinic-management-system/
|-- pom.xml                # Dependencias Maven
|-- README.md              # Instrucciones rapidas
|-- .gitignore
|-- Dockerfile             # Imagen Docker multi-etapa
|-- docker-compose.yml     # Orquestacion Docker
|-- docs/                  # Documentacion LaTeX
|   |-- main.tex
|   |-- chapters/
|   |-- images/
|-- src/
|   |-- main/
|       |-- java/com/veterinary/
|           |-- VeterinaryApplication.java
|           |-- config/
|           |-- controller/    # 20 REST controllers
|           |-- domain/       # 20 entidades + 9 enums
|           |-- dto/          # 6 DTOs
|           |-- repository/    # 19 repositorios
```

```

| | |-- scheduler/
| | |-- security/          # 4 clases de seguridad
| | |-- service/          # 22 servicios
| |-- resources/
|     |-- application.properties
|     |-- static/
|         |-- index.html
|         |-- css/app.css
|         |-- js/{api.js, app.js, vet-extras.js}
|-- test/
    |-- java/com/veterinary/
    |   |-- VeterinarySystemIntegrationTest.java    # 15 tests
    |   |-- NewFeaturesIntegrationTest.java         # 15 tests
    |-- resources/
        |-- application.properties

```

9.10 Comandos Útiles

Comando	Descripción
<code>mvn spring-boot:run</code>	Ejecutar la aplicación en desarrollo
<code>mvn clean install</code>	Compilar y empaquetar
<code>mvn test</code>	Ejecutar pruebas unitarias y de integración
<code>mvn test -Dtest=ClaseTest#metodo</code>	Ejecutar prueba específica
<code>docker-compose up -d</code>	Desplegar con Docker Compose
<code>docker-compose logs -f app</code>	Ver logs de la aplicación
<code>java -jar target/*.jar</code>	Ejecutar JAR compilado
<code>sudo systemctl start petclinic</code>	Iniciar servicio systemd

9.11 Resolución de Problemas

9.11.1 Error de Conexión a Base de Datos

Verificar que PostgreSQL esté en ejecución y accesible:

```

1 sudo systemctl status postgresql
2 psql -U veterinary_user -d veterinary_db -h localhost

```

Listing 9.12: Verificar conexión PostgreSQL

Si el error persiste, verificar que `pg_hba.conf` permita conexiones locales con autenticación por contraseña.

9.11.2 Puerto Ocupado

Si el puerto 8090 ya está en uso, se puede cambiar en `application.properties` o vía variable de entorno:

```
1 server.port=8091
2 # O via entorno:
3 export SERVER_PORT=8091
```

9.11.3 Error de Compilación

Asegurar que Java 21 esté configurado correctamente:

```
1 java -version # Debe mostrar 21.x
2 mvn -version  # Debe usar Java 21
```

9.11.4 Error de JWT

Si los tokens JWT no se validan correctamente, verificar que `JWT_SECRET` tenga al menos 256 bits (32 caracteres) y sea consistente entre reinicios de la aplicación.

9.11.5 WebSocket no conecta

Verificar que el proxy inverso (si existe) tenga configurada la actualización de protocolo WebSocket (`Upgrade` y `Connection` headers). Para Nginx, asegurar que la ubicación `/ws` tenga las directivas `proxy_set_header Upgrade` y `proxy_set_header Connection upgrade`.